

BUS_ROUTE_PERFORMANCE_EDA

import unittest

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

✓ 7.9s Python

```
1 df=pd.read_csv(r"C:\Users\Administrator\Downloads\pune_traffic.csv")
2 df.head()
```

✓ 0.1s Python

	route_id	route_name	start_stop	end_stop	distance_km	scheduled_time_min	actual_time_min	avg_speed_kmph	day_of_week	time_of_day	...	risk_level
0	PMPML-19	Express	Hadapsar	Katraj	12.82	42.0	17	22.78	Thu	Morning	...	Low
1	PMPML-33	Local	Baner	Hadapsar	16.90	72.0	39	11.77	Sun	Morning	...	Low
2	PMPML-25	Local	Kharadi	Shivaji Nagar	18.87	81.0	54	32.78	Sat	Night	...	Low
3	PMPML-40	Shuttle	Baner	Aundh	4.31	79.0	18	34.08	Sun	Night	...	Low
4	PMPML-33	Express	Baner	Kothrud	6.83	82.0	59	40.81	Tue	Night	...	Low

5 rows × 25 columns

peak_hour_flag	ticket_price_inr	occupancy_percent	bus_type	driver_experience_years	num_stops_skipped	fuel_consumption_liters	complaint_count	delay_min
1.0	36.02	62.96	Non-AC	18.0	4.0	19.66	3	-25
0.0	40.54	91.63	Electric	9.0	0.0	14.73	0	-33
1.0	25.24	54.42	AC	8.0	3.0	38.29	5	-27
0.0	27.14	99.92	Non-AC	3.0	1.0	35.25	0	-61
0.0	25.07	66.87	Non-AC	7.0	4.0	5.75	1	-23

basic dataset information

```
1 df.info()
4] ✓ 0.0s
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1001 entries, 0 to 1000

Data columns (total 25 columns):

#	Column	Non-Null Count	Dtype
0	route_id	1001 non-null	object
1	route_name	1000 non-null	object
2	start_stop	1000 non-null	object
3	end_stop	1000 non-null	object
4	distance_km	1001 non-null	float64
5	scheduled_time_min	999 non-null	float64

6	actual_time_min	1001 non-null	int64
7	avg_speed_kmph	1000 non-null	float64
8	day_of_week	998 non-null	object
9	time_of_day	1000 non-null	object
10	passenger_count	999 non-null	float64
11	weather	1001 non-null	object
12	traffic_signal_count	1000 non-null	float64
13	stop_count	999 non-null	float64
14	speed_reduction_index	1001 non-null	float64
15	risk_level	999 non-null	object
16	peak_hour_flag	1000 non-null	float64
17	ticket_price_inr	998 non-null	float64
18	occupancy_percent	1000 non-null	float64
19	bus_type	1000 non-null	object

...

23	complaint_count	1001 non-null	int64
24	delay_min	1001 non-null	int64

dtypes: float64(13), int64(3), object(9)

memory usage: 195.6+ KB

Output is truncated. View as a [scr](#)

```
1 df.shape
```

✓ 0.0s

(1001, 25)

STATISTICS

```
1 df.describe()
```

✓ 0.0s

	distance_km	scheduled_time_min	actual_time_min	avg_speed_kmph	passenger_count	traffic_signal_count	stop_count	speed_reduction_index	peak_hour_fl
count	1001.000000	999.00000	1001.000000	1000.000000	999.000000	1000.000000	999.000000	1001.000000	1000.0000
mean	13.608472	49.82983	63.868132	27.674670	41.554555	7.017000	11.101101	0.510969	0.4940
std	6.653951	22.89479	30.829217	9.994199	21.464534	4.280775	4.899265	0.289298	0.5002
min	2.030000	10.00000	10.000000	10.030000	5.000000	0.000000	3.000000	0.000000	0.0000
25%	7.660000	29.00000	38.000000	19.177500	24.000000	3.000000	7.000000	0.260000	0.0000
50%	13.840000	51.00000	63.000000	27.750000	42.000000	7.000000	11.000000	0.530000	0.0000
75%	19.340000	70.00000	91.000000	36.257500	59.000000	11.000000	15.000000	0.760000	1.0000
max	24.970000	89.00000	119.000000	44.960000	79.000000	14.000000	19.000000	1.000000	1.0000

peak_hour_flag	ticket_price_inr	occupancy_percent	driver_experience_years	num_stops_skipped	fuel_consumption_liters	complaint_count	delay_min
1000.000000	998.000000	1000.000000	1000.00000	1000.000000	1001.000000	1001.000000	1001.000000
0.494000	29.701613	60.188450	11.92100	2.014000	22.706953	4.602398	13.997003
0.500214	11.453610	22.980922	7.15975	1.423316	10.164586	2.883011	38.163137
0.000000	10.320000	20.050000	0.00000	0.000000	5.000000	0.000000	-73.000000
0.000000	19.760000	40.517500	6.00000	1.000000	14.230000	2.000000	-13.000000
0.000000	29.895000	60.140000	12.00000	2.000000	22.770000	5.000000	12.000000
1.000000	39.400000	79.955000	18.00000	3.000000	31.850000	7.000000	41.000000
1.000000	49.950000	99.920000	24.00000	4.000000	39.960000	9.000000	107.000000

MISSING VALUES

```
1 df.isnull().sum()
```

✓ 0.0s

route_id	0
route_name	1
start_stop	1
end_stop	1
distance_km	0
scheduled_time_min	2
actual_time_min	0
avg_speed_kmph	1
day_of_week	3
time_of_day	1
passenger_count	2
weather	0
traffic_signal_count	1
stop_count	2
speed_reduction_index	0
risk_level	2
peak_hour_flag	1
ticket_price_inr	3
occupancy_percent	1
bus_type	1
driver_experience_years	1

num_stops_skipped 1
fuel_consumption_liters 0
complaint_count 0
delay_min 0
dtype: int64

```
1 df['delay_min'] = df['delay_min'].fillna(df['delay_min'].median())
2 df['avg_speed_kmph'] = df['avg_speed_kmph'].fillna(df['avg_speed_kmph'].mean())
3 df['weather'] = df['weather'].fillna(df['weather'].mode()[0])
4 df['risk_level'] = df['risk_level'].fillna(df['risk_level'].mode()[0])
```

✓ 0.0s Python

```
1 df.count()
```

✓ 0.0s Python

route_id 1001
route_name 1000
start_stop 1000
end_stop 1000
distance_km 1001
scheduled_time_min 999
actual_time_min 1001
avg_speed_kmph 1001
day_of_week 998
time_of_day 1000
passenger_count 999
weather 1001
traffic_signal_count 1000
stop_count 999
speed_reduction_index 1001

risk_level	1001
peak_hour_flag	1000
ticket_price_inr	998
occupancy_percent	1000
bus_type	1000
driver_experience_years	1000
num_stops_skipped	1000
fuel_consumption_liters	1001
complaint_count	1001
delay_min	1001

dtype: int64

after analyzing the data what we can conclude is all the data types of all columns are correctly mapped and no null value found in any column `

```
1 columnsname =df.columns
```

```
1 int_columns=[]
2 for column in df.columns:
3     isinstance(df[column].dtype,(np.int64,np.float64))
4     int_columns.append(column)
```

```
1 int_columns
```

```
['route_id',
 'route_name',
 'start_stop',
 'end_stop',
```

'distance_km',
'scheduled_time_min',
'actual_time_min',
'avg_speed_kmph',
'day_of_week',
'time_of_day',
'passenger_count',
'weather',
'traffic_signal_count',
'stop_count',
'speed_reduction_index',
'risk_level',
'peak_hour_flag',
'ticket_price_inr',
'occupancy_percent',
'bus_type',
'driver_experience_years',
'num_stops_skipped',
'fuel_consumption_liters',
'complaint_count',
'delay_min']


```
1 targest_columns=[]
2 for col in df.columns:
3     if df[col].dtypes==object:
4         targest_columns.append(col)
5
```

```
1 targest_columns
```

```
['route_id',
 'route_name',
 'start_stop',
 'end_stop',
 'day_of_week',
 'time_of_day',
 'weather',
 'risk_level',
 'bus_type']
```

risk_level wise performance analysis

```
1 df["risk_level"].describe()
```

count 1001

unique 3

top Low

freq 515

Name: risk_level, dtype: object

```
1 df.groupby("risk_level")["route_id"].count()
```

risk_level

High 197

Low 515

Medium 289

Name: route_id, dtype: int64

1 df[df["risk_level"]=="High"].describe()

Python

	distance_km	scheduled_time_min	actual_time_min	avg_speed_kmph	passenger_count	traffic_signal_count	stop_count	speed_reduction_index	peak_hour_fl
count	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.0000
mean	13.645178	49.908629	65.883249	28.464772	43.761421	6.771574	10.939086	0.542995	0.4923
std	6.602713	22.138750	32.451244	9.709102	21.395664	4.044851	4.875631	0.287050	0.5012
min	2.200000	10.000000	10.000000	10.600000	5.000000	0.000000	3.000000	0.000000	0.0000
25%	8.370000	31.000000	38.000000	20.470000	27.000000	3.000000	7.000000	0.320000	0.0000
50%	13.450000	51.000000	64.000000	27.880000	45.000000	7.000000	11.000000	0.560000	0.0000
75%	19.130000	69.000000	96.000000	36.440000	62.000000	10.000000	15.000000	0.780000	1.0000
max	24.950000	89.000000	119.000000	44.960000	79.000000	14.000000	19.000000	1.000000	1.0000

duction_index	peak_hour_flag	ticket_price_inr	occupancy_percent	driver_experience_years	num_stops_skipped	fuel_consumption_liters	complaint_count	delay_min
197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000	197.000000
0.542995	0.492386	29.400355	63.019797	12.111675	2.005076	23.220711	4.888325	15.974619
0.287050	0.501216	10.668560	23.525815	6.905168	1.430347	9.700885	2.879859	39.749013
0.000000	0.000000	10.380000	20.210000	0.000000	0.000000	5.040000	0.000000	-72.000000
0.320000	0.000000	20.370000	42.570000	6.000000	1.000000	15.740000	3.000000	-11.000000
0.560000	0.000000	30.140000	64.730000	12.000000	2.000000	23.810000	5.000000	17.000000
0.780000	1.000000	37.090000	84.500000	18.000000	3.000000	31.720000	7.000000	45.000000
1.000000	1.000000	49.940000	99.590000	24.000000	4.000000	39.950000	9.000000	96.000000

```
1 df[df["risk_level"]=="Low"].describe()
```

	distance_km	scheduled_time_min	actual_time_min	avg_speed_kmph	passenger_count	traffic_signal_count	stop_count	speed_reduction_index
count	515.000000	514.000000	515.000000	515.000000	514.000000	514.000000	513.000000	515.000000
mean	13.922835	49.363813	62.316505	27.523709	41.416342	7.007782	11.290448	0.508544
std	6.626259	23.052824	30.315636	10.021752	21.306494	4.298776	5.058587	0.290052
min	2.030000	10.000000	10.000000	10.070000	5.000000	0.000000	3.000000	0.000000
25%	7.860000	29.000000	37.000000	18.760000	24.000000	3.000000	7.000000	0.245000
50%	14.620000	50.000000	62.000000	27.600000	42.000000	7.000000	11.000000	0.520000
75%	19.480000	69.000000	87.000000	36.015000	59.000000	11.000000	16.000000	0.765000
max	24.970000	89.000000	119.000000	44.900000	79.000000	14.000000	19.000000	1.000000

duction_index	peak_hour_flag	ticket_price_inr	occupancy_percent	driver_experience_years	num_stops_skipped	fuel_consumption_liters	complaint_count	delay_min
515.000000	514.000000	514.000000	515.000000	515.000000	514.000000	515.000000	515.000000	515.000000
0.508544	0.464981	30.265253	59.200990	11.838835	2.056420	22.354990	4.530097	12.935922
0.290052	0.499258	11.751802	23.135809	7.347890	1.417905	10.286141	2.892457	38.080727
0.000000	0.000000	10.320000	20.050000	0.000000	0.000000	5.080000	0.000000	-72.000000
0.245000	0.000000	19.972500	38.310000	5.000000	1.000000	13.690000	2.000000	-14.000000
0.520000	0.000000	30.570000	59.550000	12.000000	2.000000	22.020000	5.000000	11.000000
0.765000	1.000000	40.595000	79.545000	18.000000	3.000000	31.535000	7.000000	40.000000
1.000000	1.000000	49.930000	99.920000	24.000000	4.000000	39.950000	9.000000	107.000000

```
1 df[df["risk_level"]=="Medium"].describe()
```

Python

	distance_km	scheduled_time_min	actual_time_min	avg_speed_kmph	passenger_count	traffic_signal_count	stop_count	speed_reduction_index	peak_hour_fl
count	289.000000	288.000000	289.000000	289.000000	288.000000	289.000000	289.000000	289.000000	289.0000
mean	13.023253	50.607639	65.259516	27.405103	40.291667	7.200692	10.875433	0.493460	0.5467
std	6.721594	23.175340	30.564795	10.125648	21.749885	4.408940	4.623129	0.288718	0.4986
min	2.040000	11.000000	10.000000	10.030000	5.000000	0.000000	3.000000	0.020000	0.0000
25%	6.880000	30.750000	42.000000	19.330000	21.000000	3.000000	7.000000	0.250000	0.0000
50%	13.330000	51.500000	67.000000	27.740000	38.500000	7.000000	11.000000	0.500000	1.0000
75%	18.590000	70.250000	91.000000	35.970000	58.000000	11.000000	15.000000	0.730000	1.0000
max	24.900000	89.000000	119.000000	44.780000	79.000000	14.000000	19.000000	1.000000	1.0000

duction_index	peak_hour_flag	ticket_price_inr	occupancy_percent	driver_experience_years	num_stops_skipped	fuel_consumption_liters	complaint_count	delay_min
289.000000	289.000000	287.000000	288.00000	288.000000	289.000000	289.000000	289.000000	289.000000
0.493460	0.546713	28.898955	60.01750	11.937500	1.944637	22.983945	4.536332	14.539792
0.288718	0.498677	11.415455	22.23562	7.009917	1.430222	10.265947	2.866151	37.264995
0.020000	0.000000	10.320000	20.39000	0.000000	0.000000	5.000000	0.000000	-73.000000
0.250000	0.000000	18.485000	41.23750	6.000000	1.000000	13.930000	2.000000	-10.000000
0.500000	1.000000	28.150000	59.82000	12.000000	2.000000	23.480000	5.000000	14.000000
0.730000	1.000000	38.085000	78.10000	18.000000	3.000000	32.140000	7.000000	43.000000
1.000000	1.000000	49.950000	99.90000	24.000000	4.000000	39.960000	9.000000	97.000000

avg_speed_kmph wise comparison

```
1 df.groupby("avg_speed_kmph").agg(  
2     {  
3         "scheduled_time_min":["min","mean","max"],  
4         "actual_time_min":["min","mean","max"],  
5         "stop_count":["min","mean","max"],  
6         "traffic_signal_count":["min","mean","max"],  
7         "speed_reduction_index":["min","mean","max"],  
8         "peak_hour_flag":["min","mean","max"],  
9         "ticket_price_inr":["min","mean","max"]  
10    }  
11 )
```

	scheduled_time_min			actual_time_min			stop_count			traffic_signal_count			speed_reduction_index			peak_hour_flag			ticket_price_inr		
	min	mean	max	min	mean	max	min	mean	max	min	...	max	min	mean	max	min	mean	max	min	mean	max
avg_speed_kmph																					
10.03	31.0	31.0	31.0	55	55.0	55	7.0	7.0	7.0	3.0	...	3.0	0.65	0.650	0.65	1.0	1.0	1.0	25.98	25.980	25.98
10.05	59.0	59.0	59.0	37	37.0	37	6.0	6.0	6.0	1.0	...	1.0	0.94	0.940	0.94	0.0	0.0	0.0	16.72	16.720	16.72
10.07	14.0	51.0	88.0	60	64.5	69	3.0	10.0	17.0	13.0	...	14.0	0.06	0.500	0.94	1.0	1.0	1.0	16.91	30.040	43.17
10.08	19.0	44.5	70.0	56	58.0	60	10.0	11.5	13.0	2.0	...	4.0	0.01	0.180	0.35	0.0	0.5	1.0	21.90	27.360	32.82
10.13	31.0	31.0	31.0	43	43.0	43	12.0	12.0	12.0	5.0	...	5.0	0.11	0.110	0.11	1.0	1.0	1.0	40.92	40.920	40.92
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
44.84	58.0	58.0	58.0	45	45.0	45	19.0	19.0	19.0	10.0	...	10.0	0.29	0.290	0.29	1.0	1.0	1.0	47.22	47.220	47.22
44.86	14.0	29.5	45.0	50	62.0	74	11.0	14.5	18.0	7.0	...	8.0	0.60	0.695	0.79	1.0	1.0	1.0	13.53	16.235	18.94
44.90	58.0	58.0	58.0	79	79.0	79	14.0	14.0	14.0	6.0	...	6.0	0.05	0.050	0.05	0.0	0.0	0.0	41.36	41.360	41.36
44.95	86.0	86.0	86.0	18	18.0	18	13.0	13.0	13.0	9.0	...	9.0	0.55	0.550	0.55	1.0	1.0	1.0	44.38	44.380	44.38
44.96	44.0	44.0	44.0	26	26.0	26	8.0	8.0	8.0	6.0	...	6.0	0.81	0.810	0.81	0.0	0.0	0.0	41.01	41.010	41.01

int_columns+tagest_columns

['route_id',

'route_name',

'start_stop',
'end_stop',
'distance_km',
'scheduled_time_min',
'actual_time_min',
'avg_speed_kmph',
'day_of_week',
'time_of_day',
'passenger_count',
'weather',
'traffic_signal_count',
'stop_count',
'speed_reduction_index',
'risk_level',
'peak_hour_flag',
'ticket_price_inr',
'occupancy_percent',
'bus_type',

```
'driver_experience_years',  
'num_stops_skipped',  
'fuel_consumption_liters',  
'complaint_count',  
'delay_min',  
...  
'day_of_week',  
'time_of_day',  
'weather',  
'risk_level',  
'bus_type']
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```
1 def get_corr_df():
2     categories_df = df.drop(int_columns + float_columns, axis=1)
3     corr_df = df[int_columns + float_columns]
4     return categories_df, corr_df
5
```

0.0s

```
1 corr_df = df.select_dtypes(include="number")
2
3 plt.figure(figsize=(12,8))
4
5 sns.heatmap(
6     corr_df.corr(),
7     annot=True,
8     cmap="coolwarm",
9     fmt=".2f"
10 )
11
12 plt.title("Correlation Matrix")
13 plt.show()
```

1.8s


```
1 positive_correlation_columns=["distance_km","scheduled_time_min","actual_time_min","avg_speed_kmph","passenger_count","delay_min"]
```

3] ✓ 0.0s

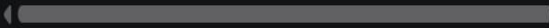
Python

```
1 int_column = df.select_dtypes(include=np.number).columns.tolist()
2 print(int_column)
```

4] ✓ 0.0s

Python

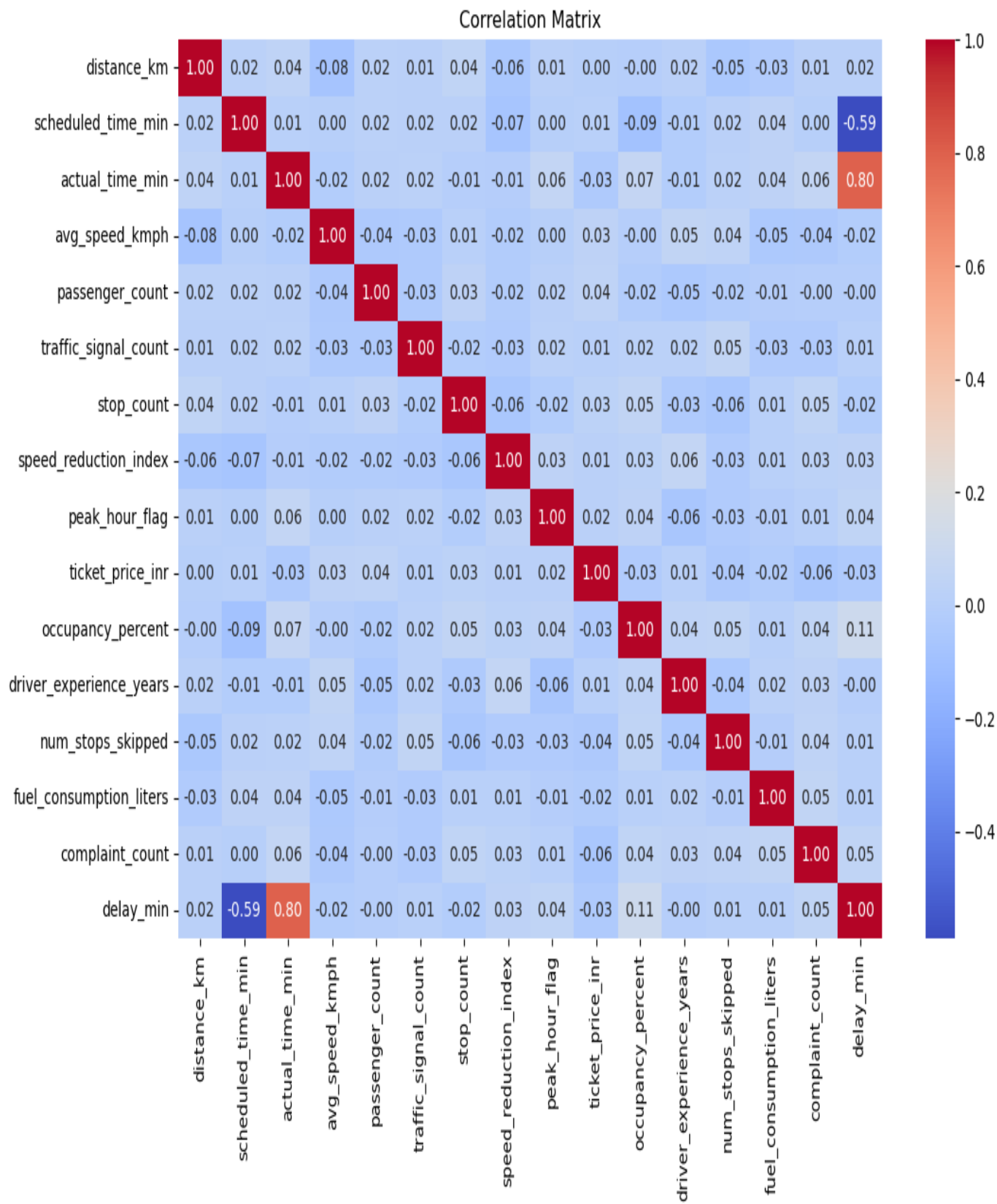
```
['distance_km', 'scheduled_time_min', 'actual_time_min', 'avg_speed_kmph', 'passenger_count', 'traffic_signal_count', 'stop_count', 'speed_reduction_in
```



```
1 for i in range(len(int_column)-1):
2     plt.scatter(df[int_column[i]], df[int_column[i+1]])
3     plt.xlabel(int_column[i])
4     plt.ylabel(int_column[i+1])
5     plt.title(f"{int_column[i]} vs {int_column[i+1]}")
6     plt.show()
```

5] ✓ 4.2s

Python



```
1 positive_correlation_columns=["distance_km","scheduled_time_min","actual_time_min","avg_speed_kmph","passenger_count","delay_min"]
```

33] ✓ 0.0s

Python

```
1 int_column = df.select_dtypes(include=np.number).columns.tolist()
2 print(int_column)
```

34] ✓ 0.0s

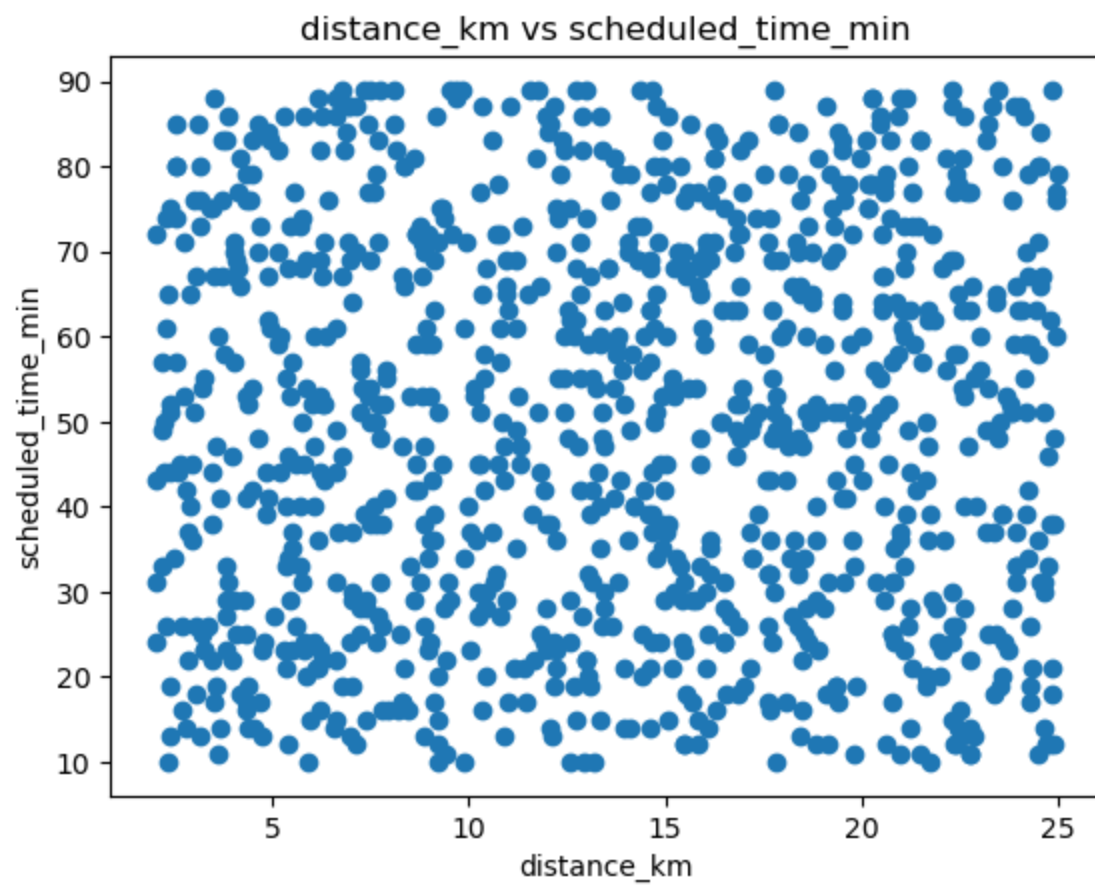
Python

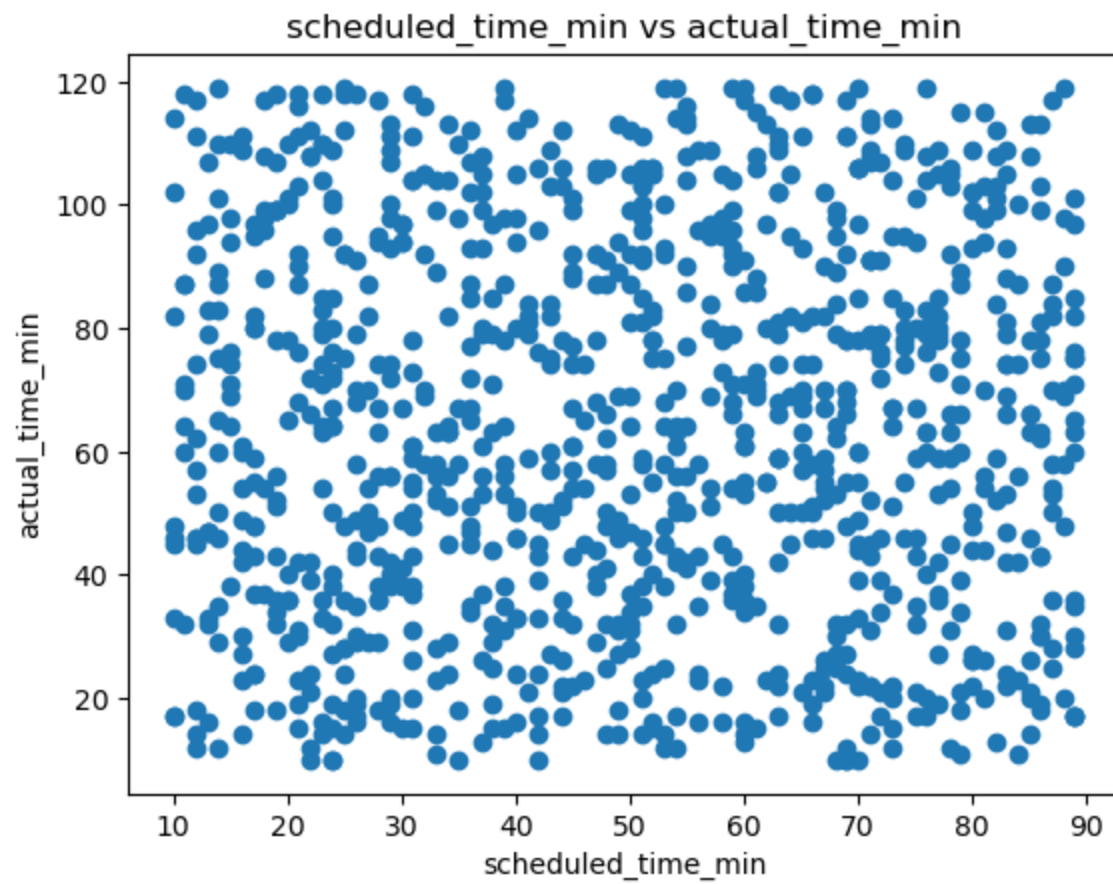
... ['distance_km', 'scheduled_time_min', 'actual_time_min', 'avg_speed_kmph', 'passenger_count', 'traffic_signal_count', 'stop_count', 'speed_reduction_in

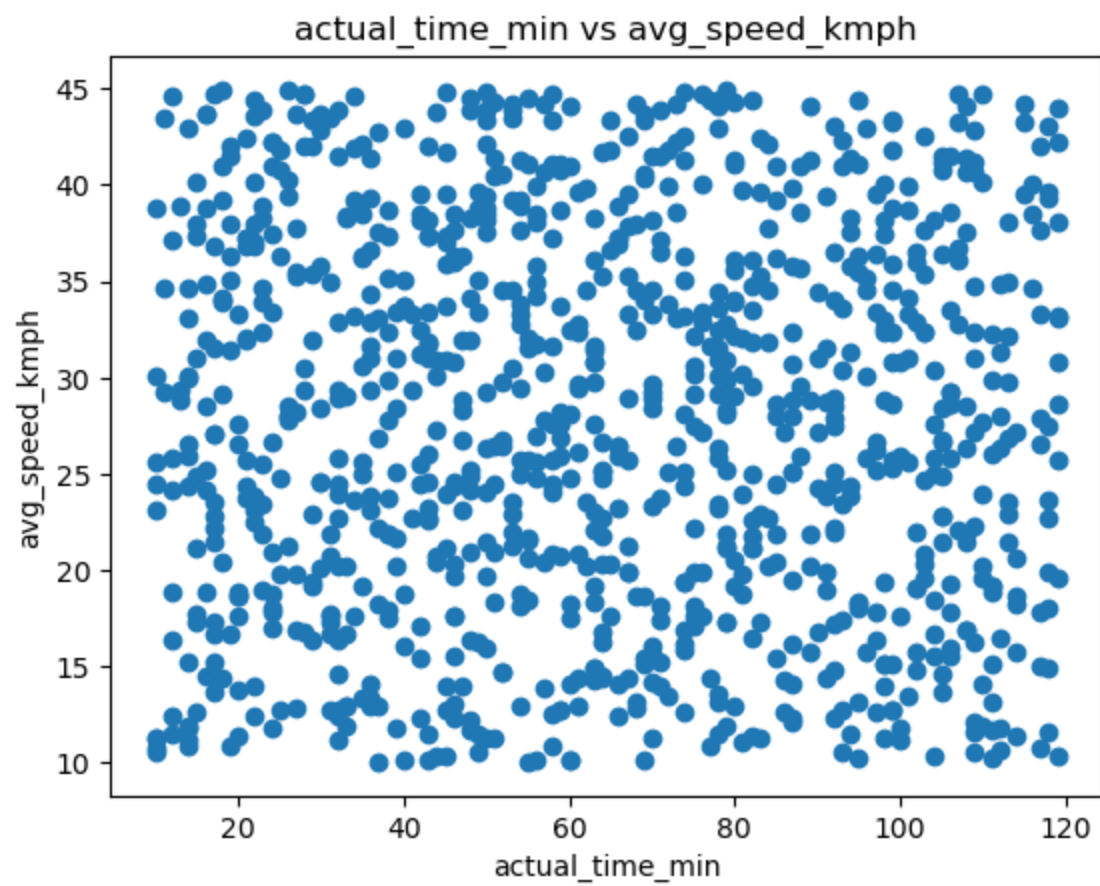
```
1 for i in range(len(int_column)-1):
2     plt.scatter(df[int_column[i]], df[int_column[i+1]])
3     plt.xlabel(int_column[i])
4     plt.ylabel(int_column[i+1])
5     plt.title(f"{int_column[i]} vs {int_column[i+1]}")
6     plt.show()
```

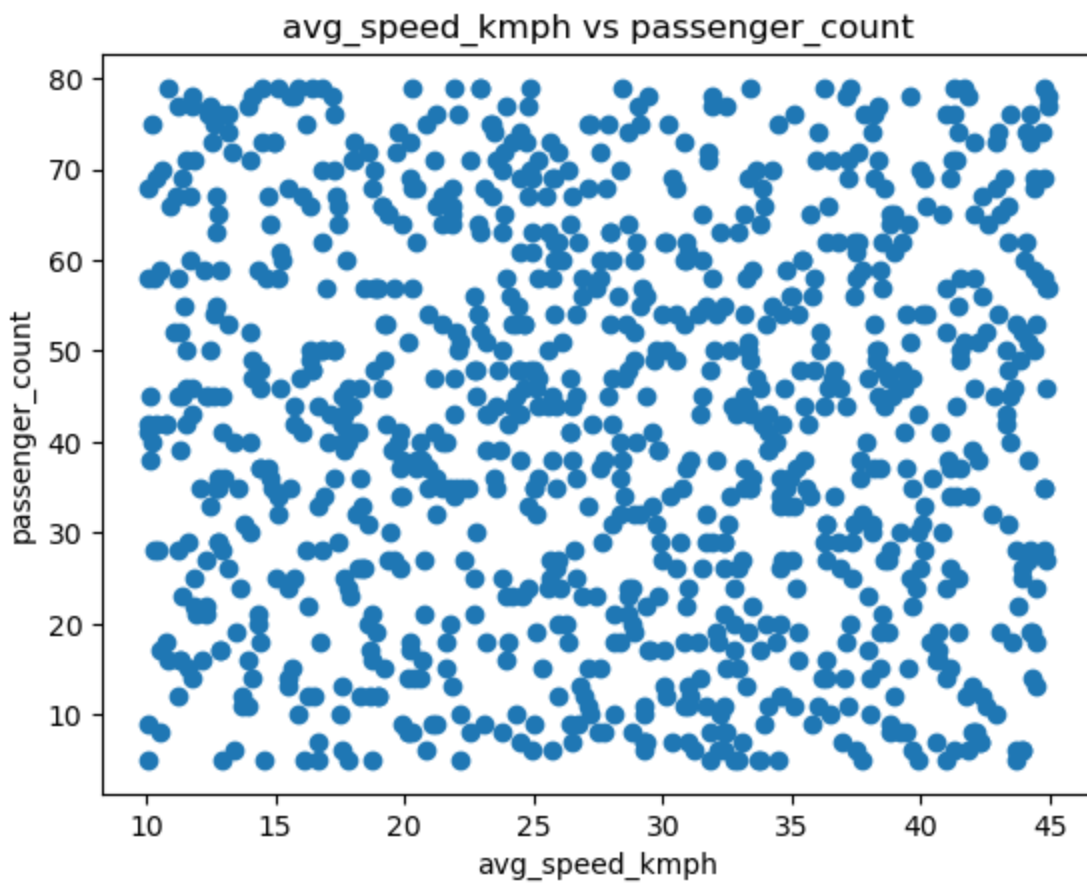
35] ✓ 4.2s

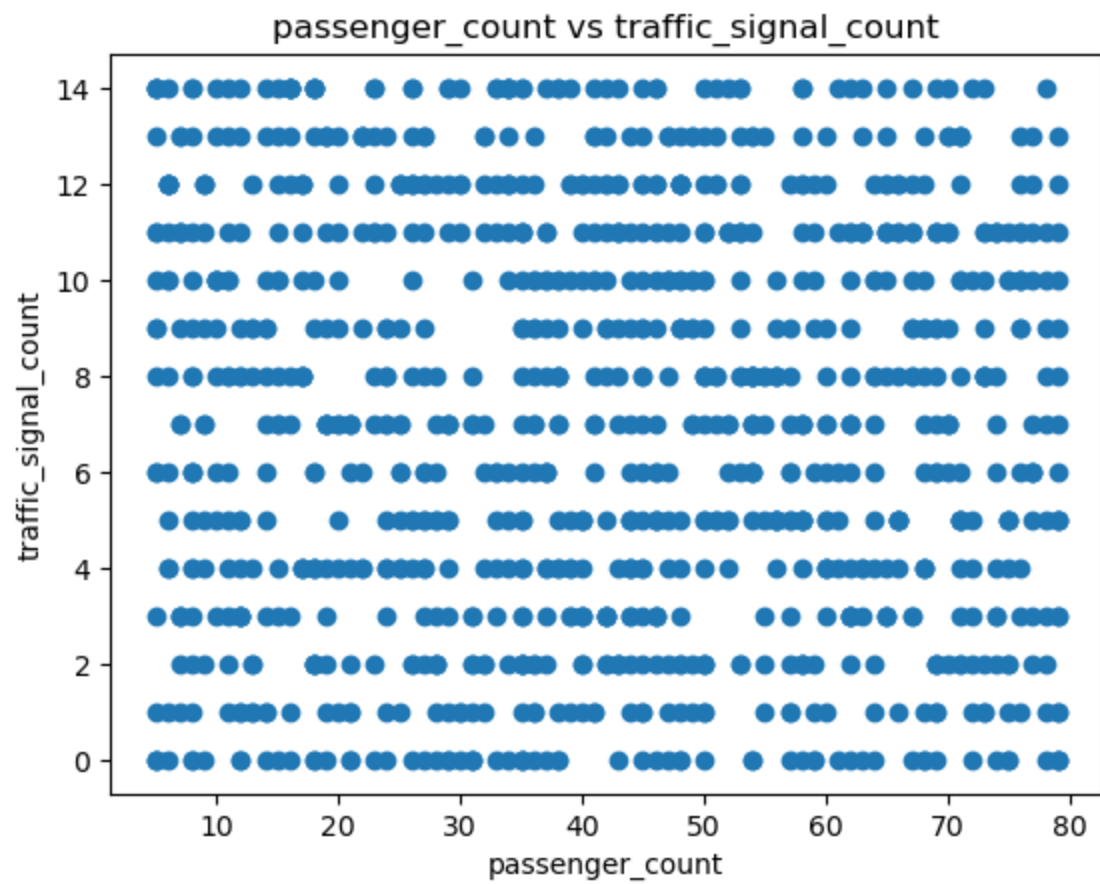
Python

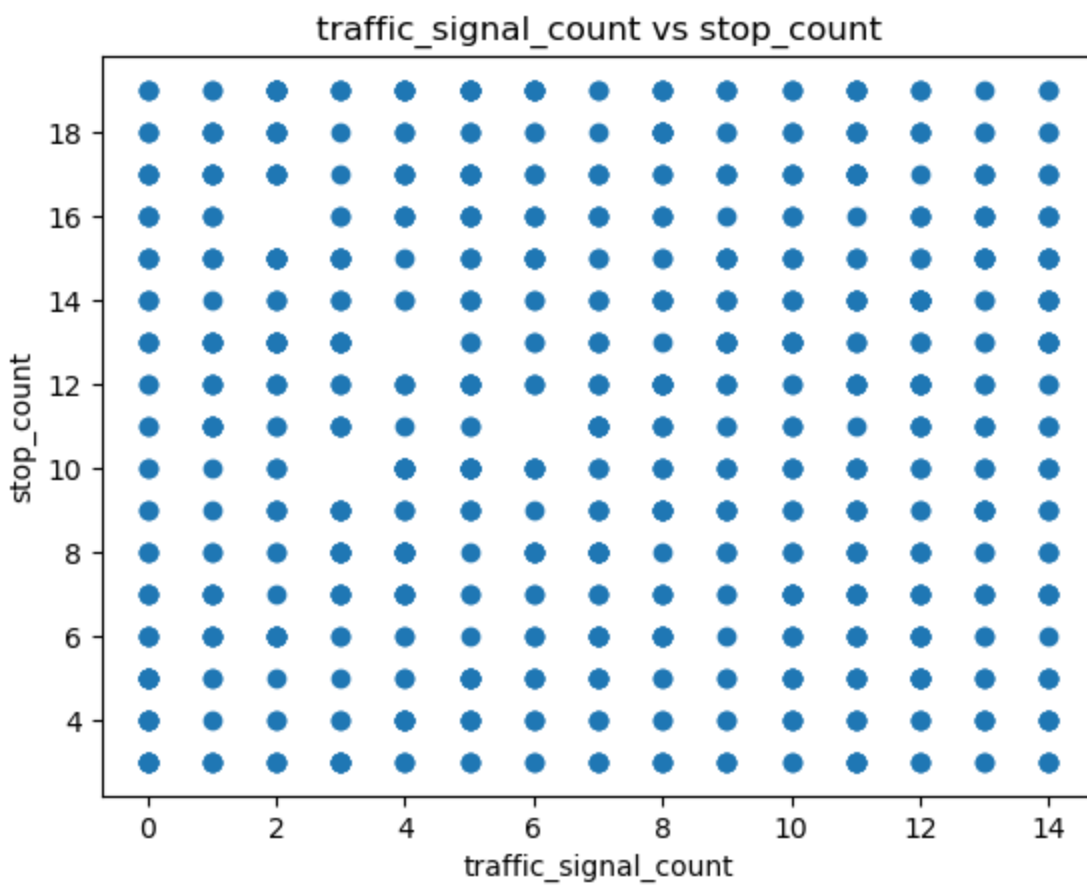


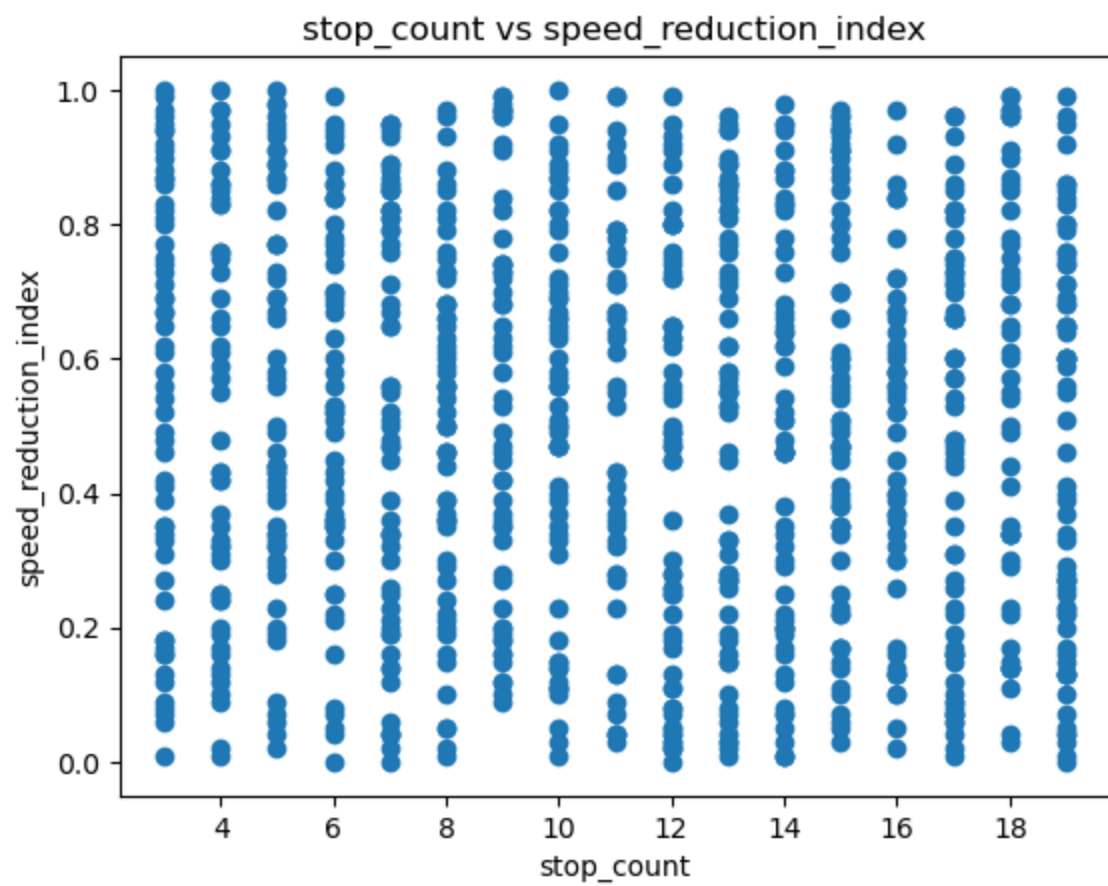


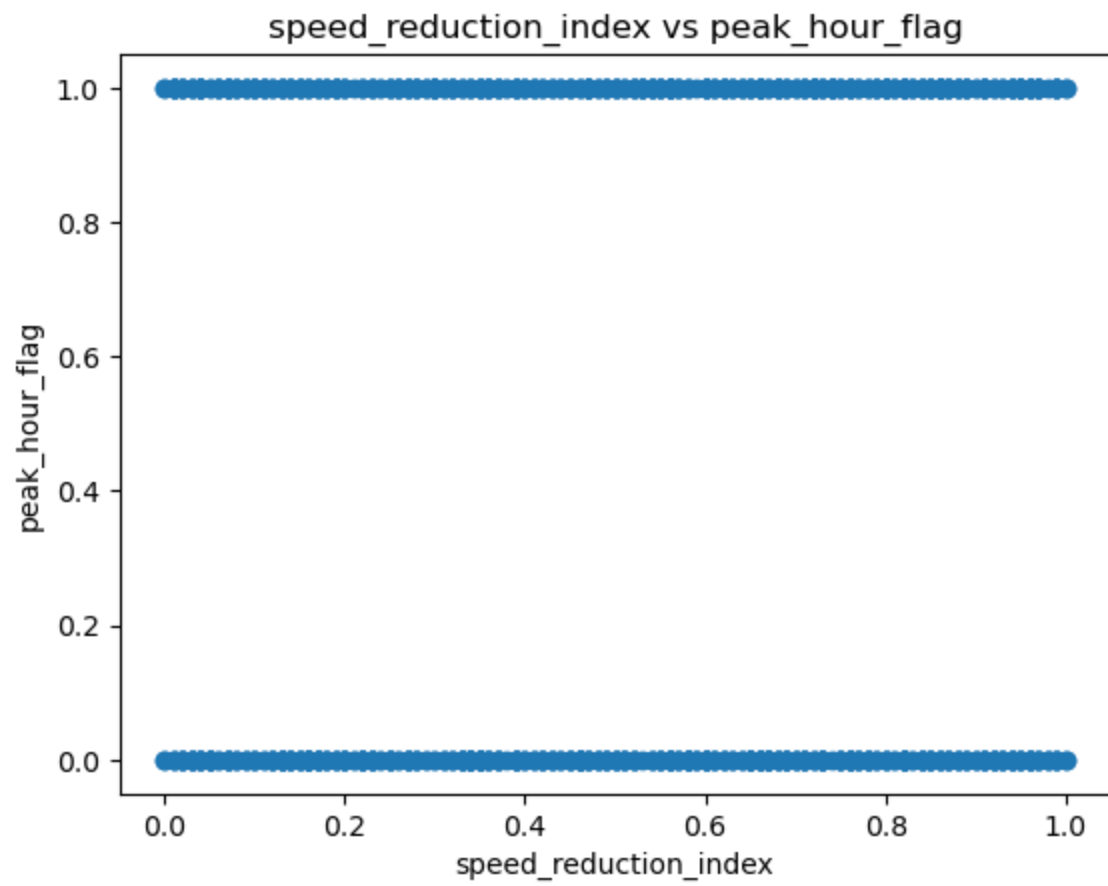


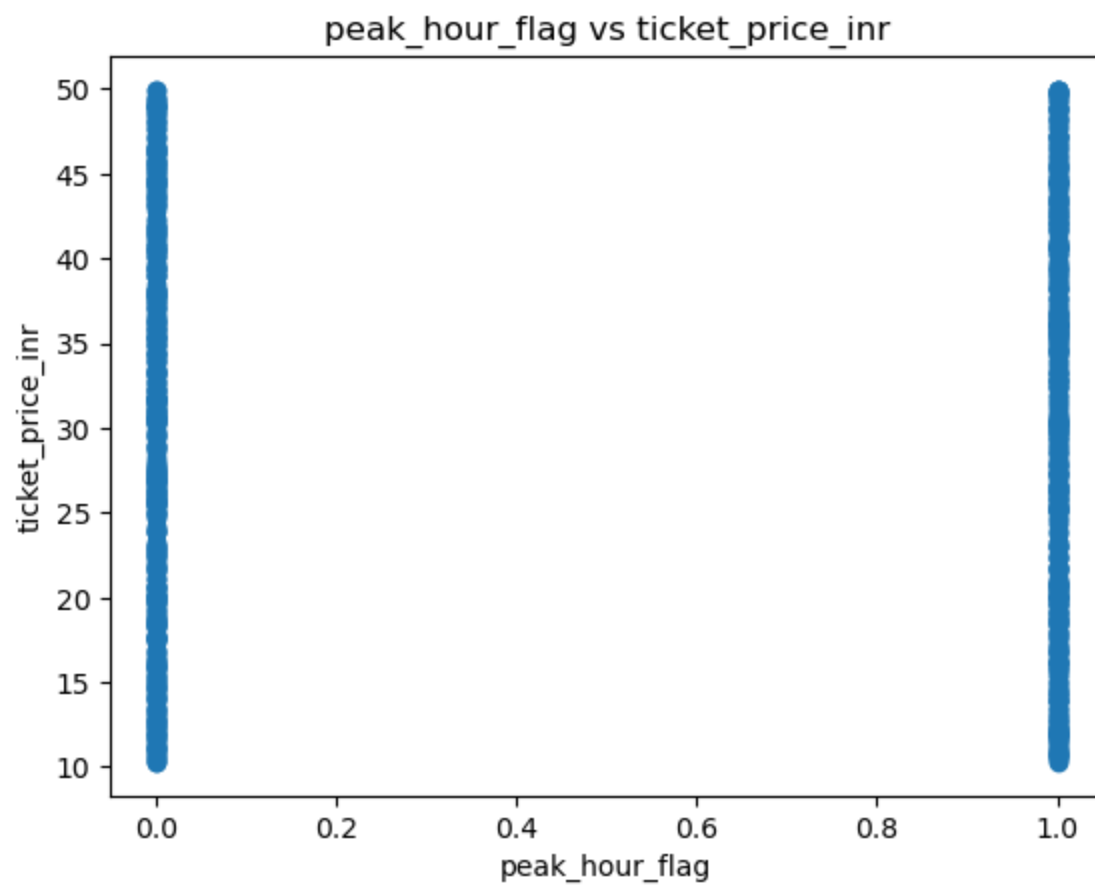


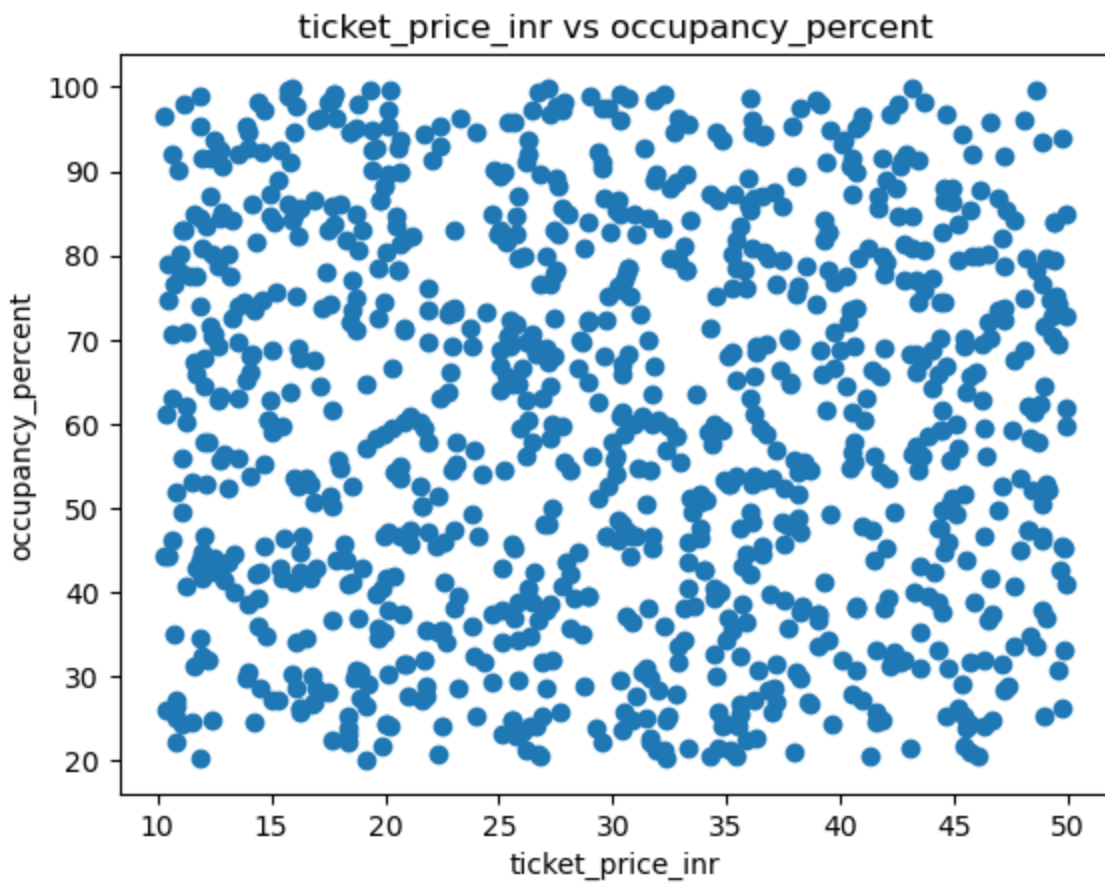


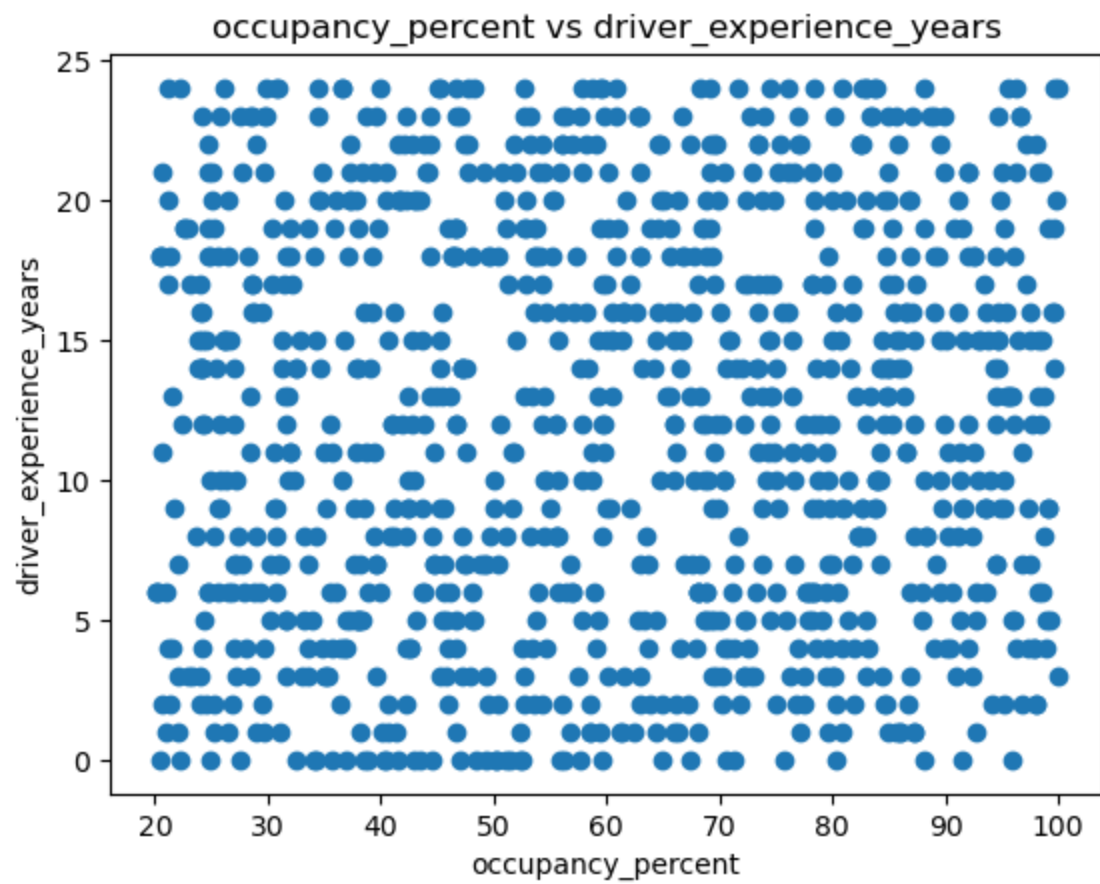


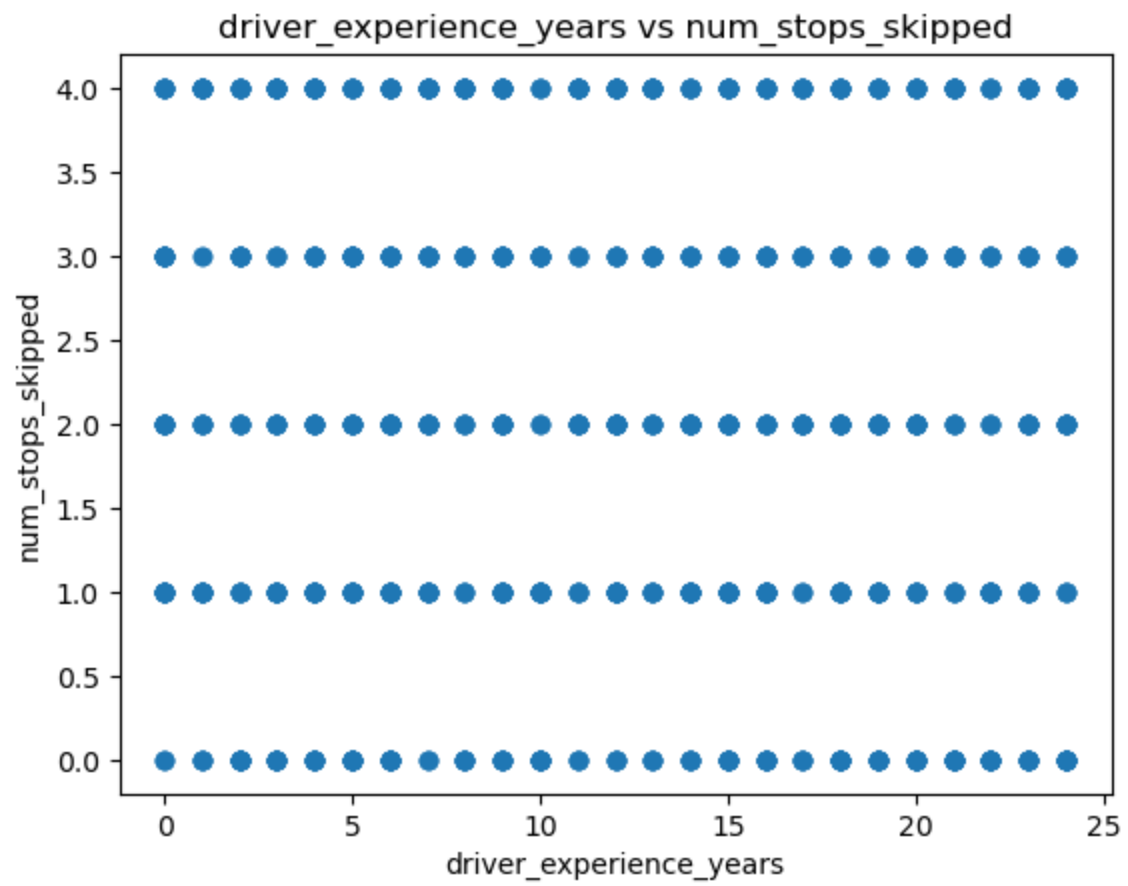


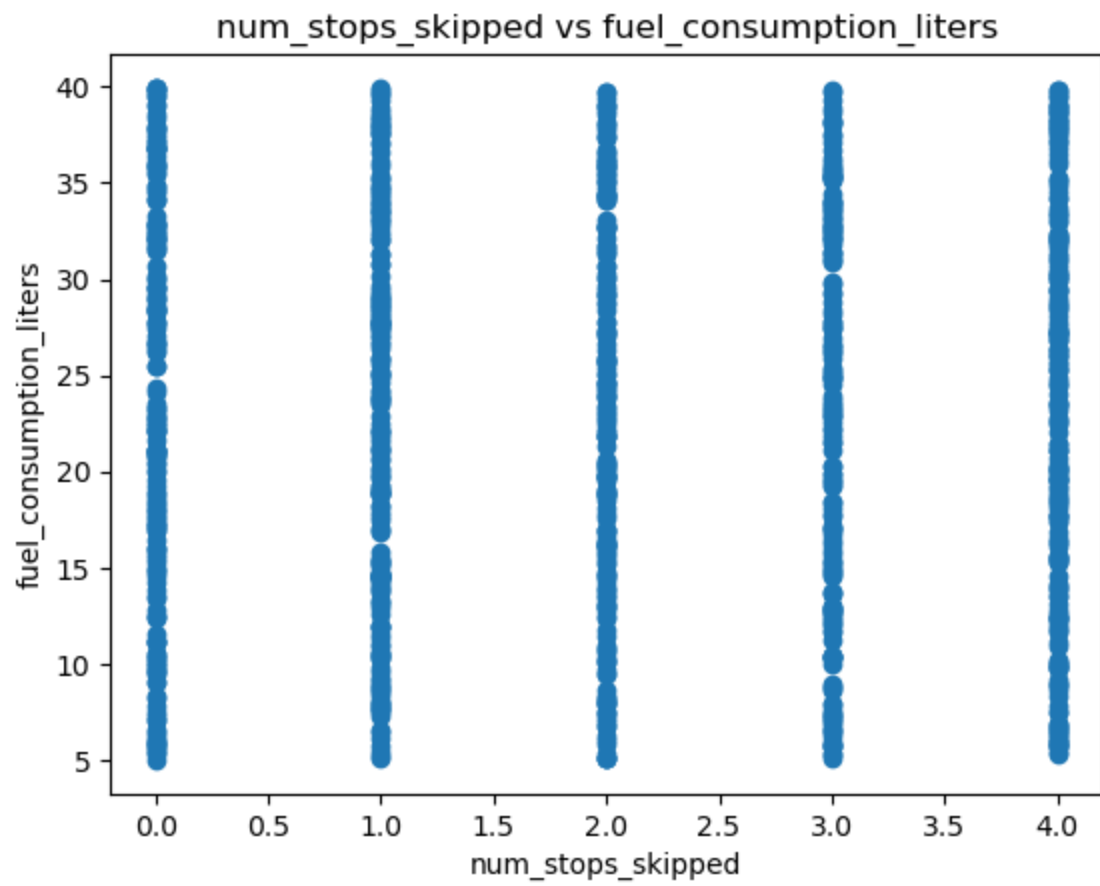


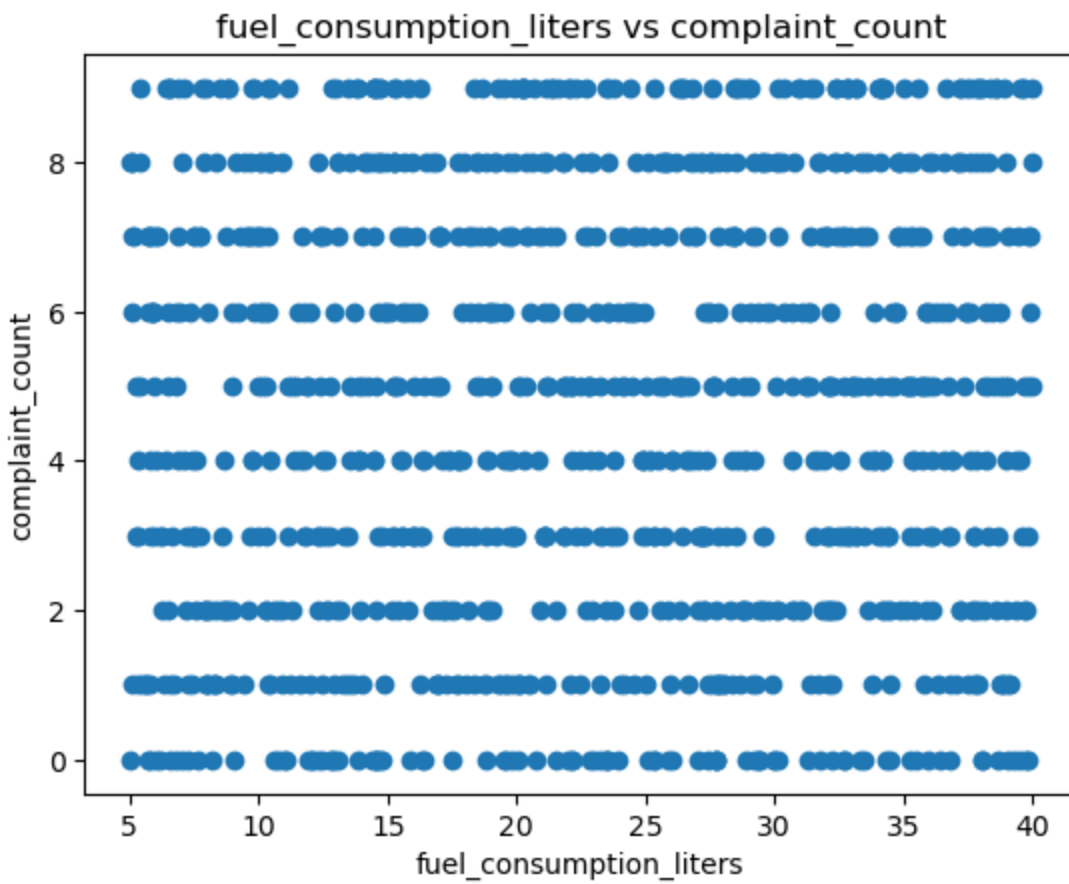


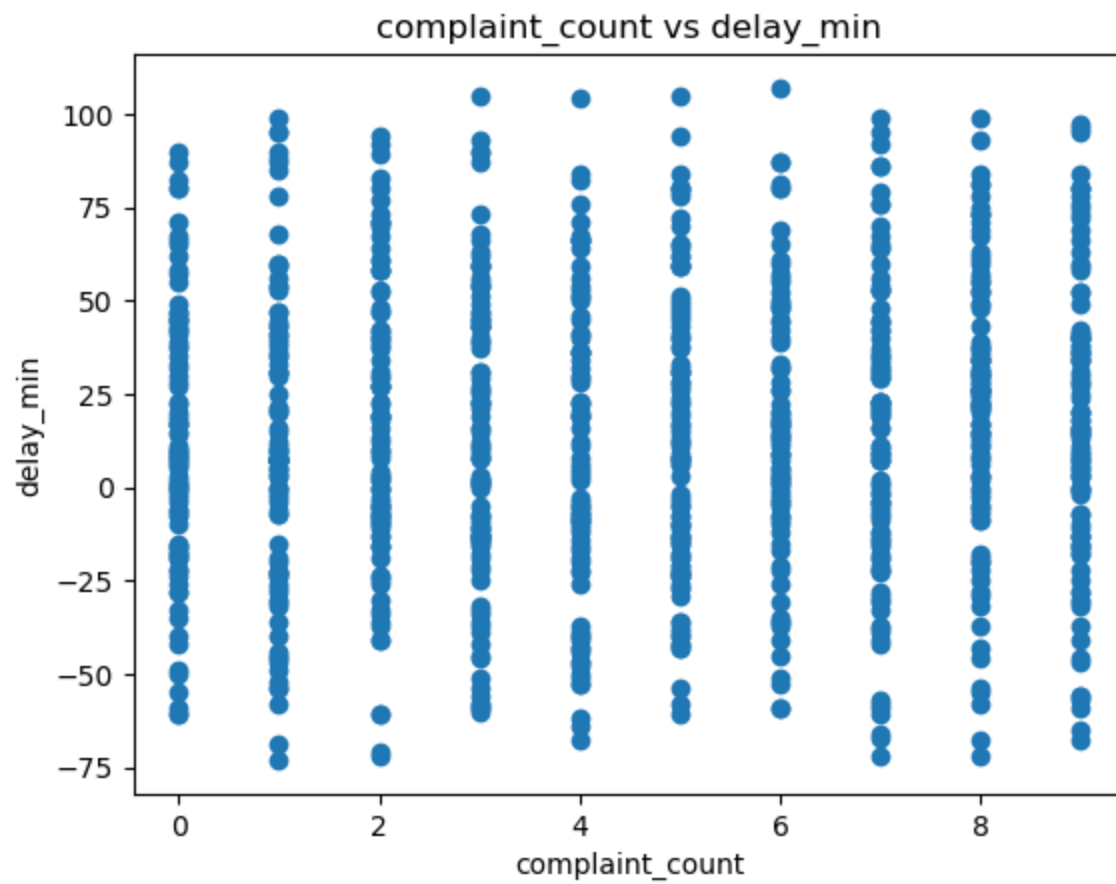










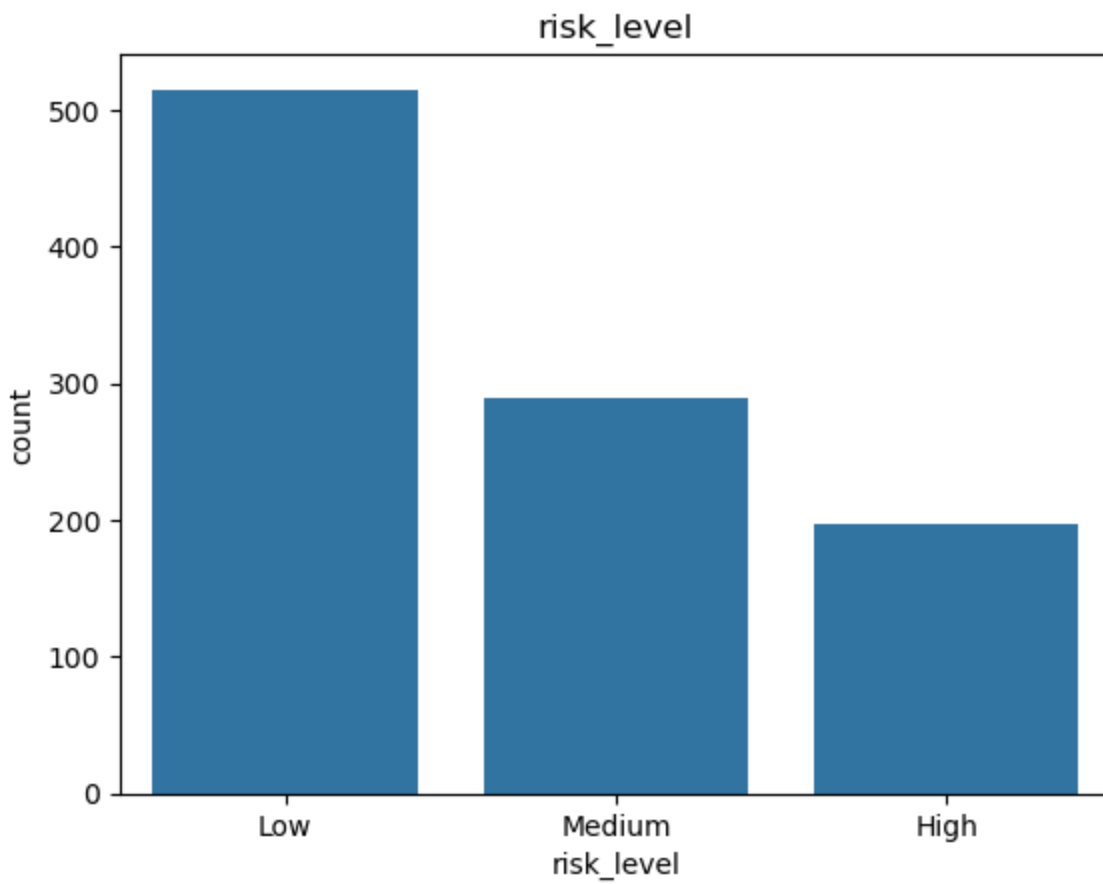


Click to add a breakpoint: `'risk_level', data=df)`

```
2 plt.title("risk_level")
```

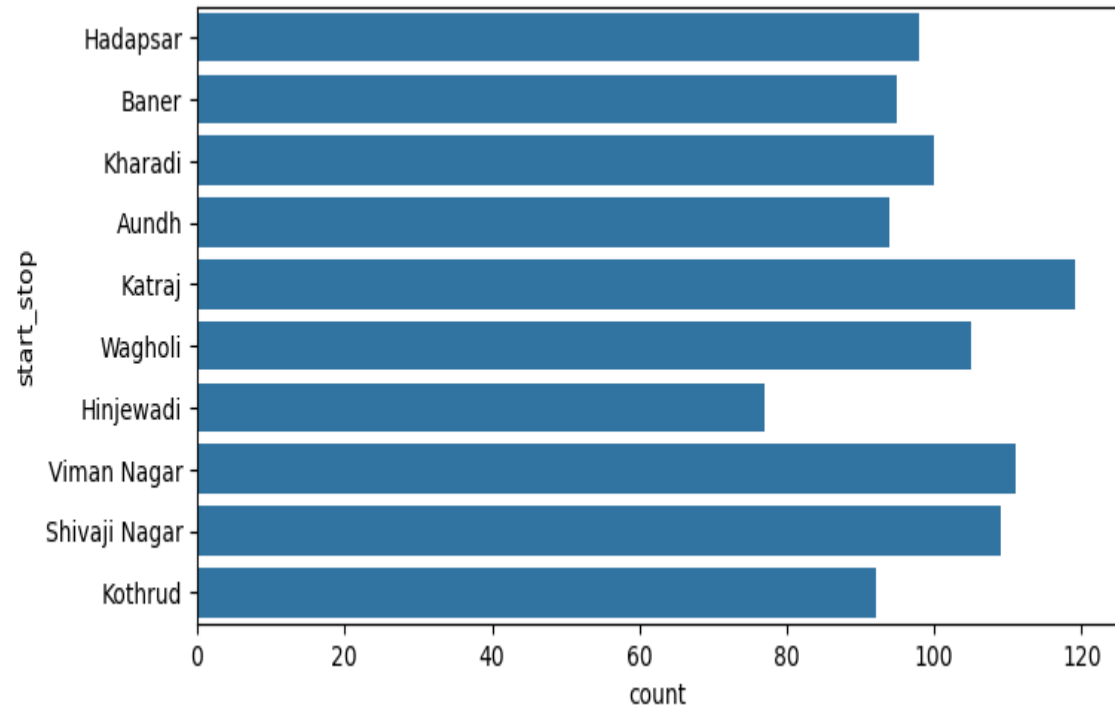
```
3 plt.show()
```

✓ 0.2s

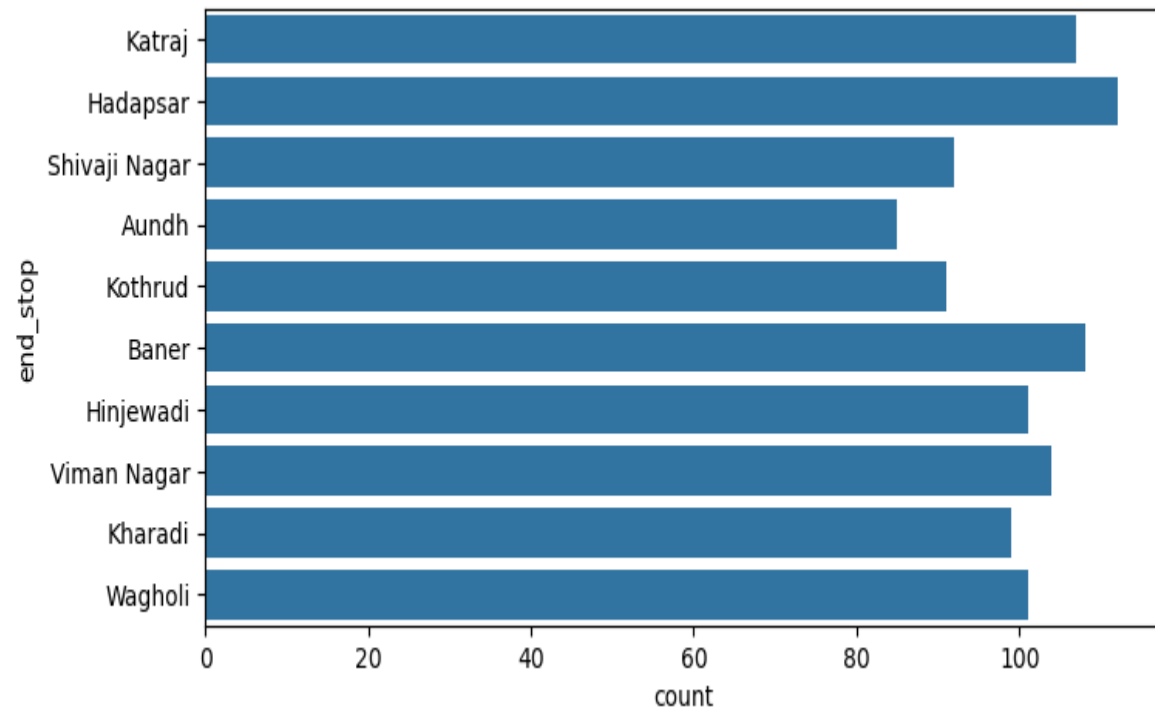


```
1 plt.figure(figsize=(8,4))
2 sns.countplot(y='start_stop',data=df)
3 plt.show()
```

✓ 0.2s

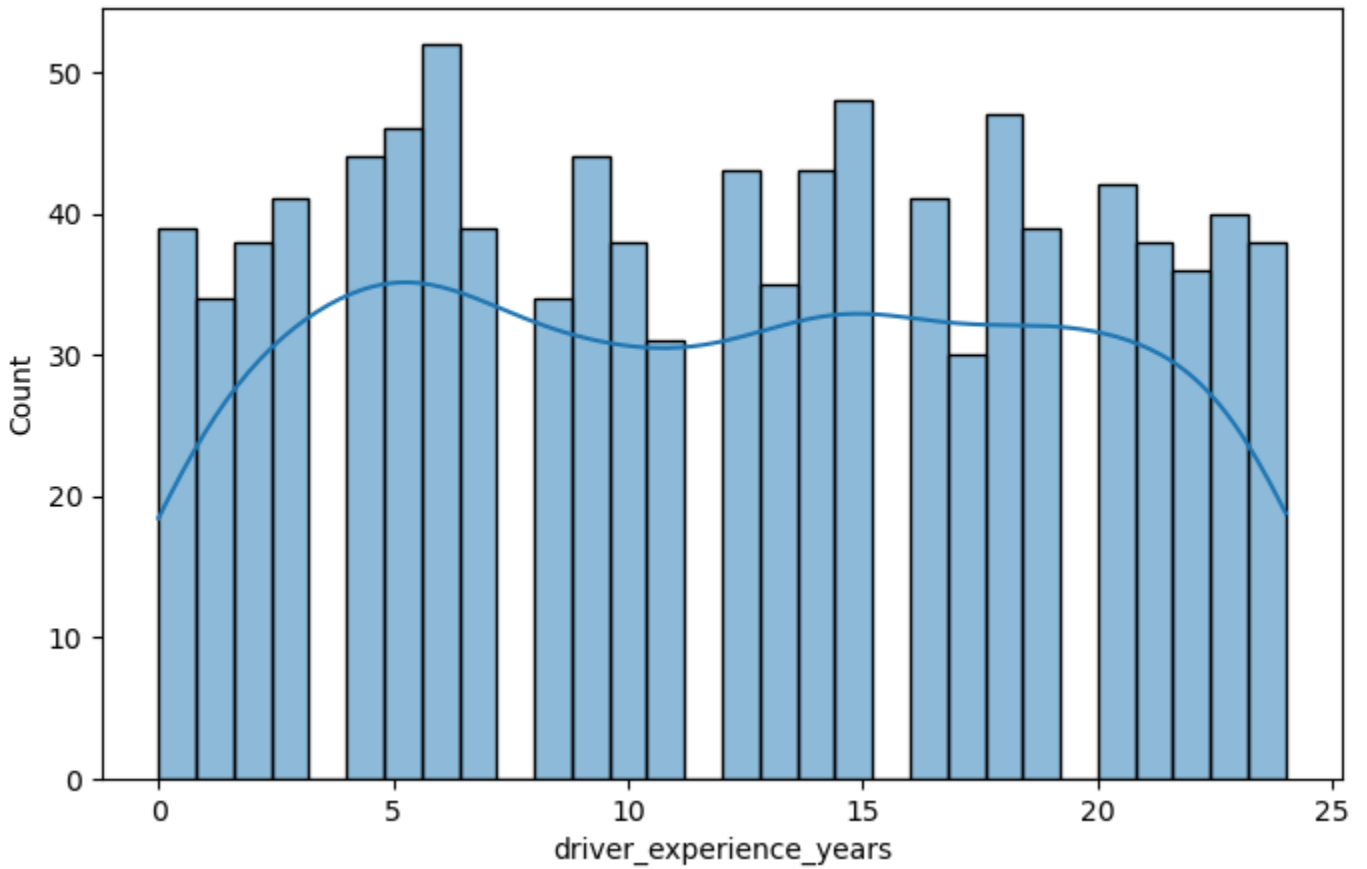


```
1 plt.figure(figsize=(8,4))
2 sns.countplot(y='end_stop',data=df)
3 plt.show()
✓ 0.2s
```



```
1 plt.figure(figsize=(8,5))
2 sns.histplot(df['driver_experience_years'],bins=30,kde=True)
3 plt.show()
```

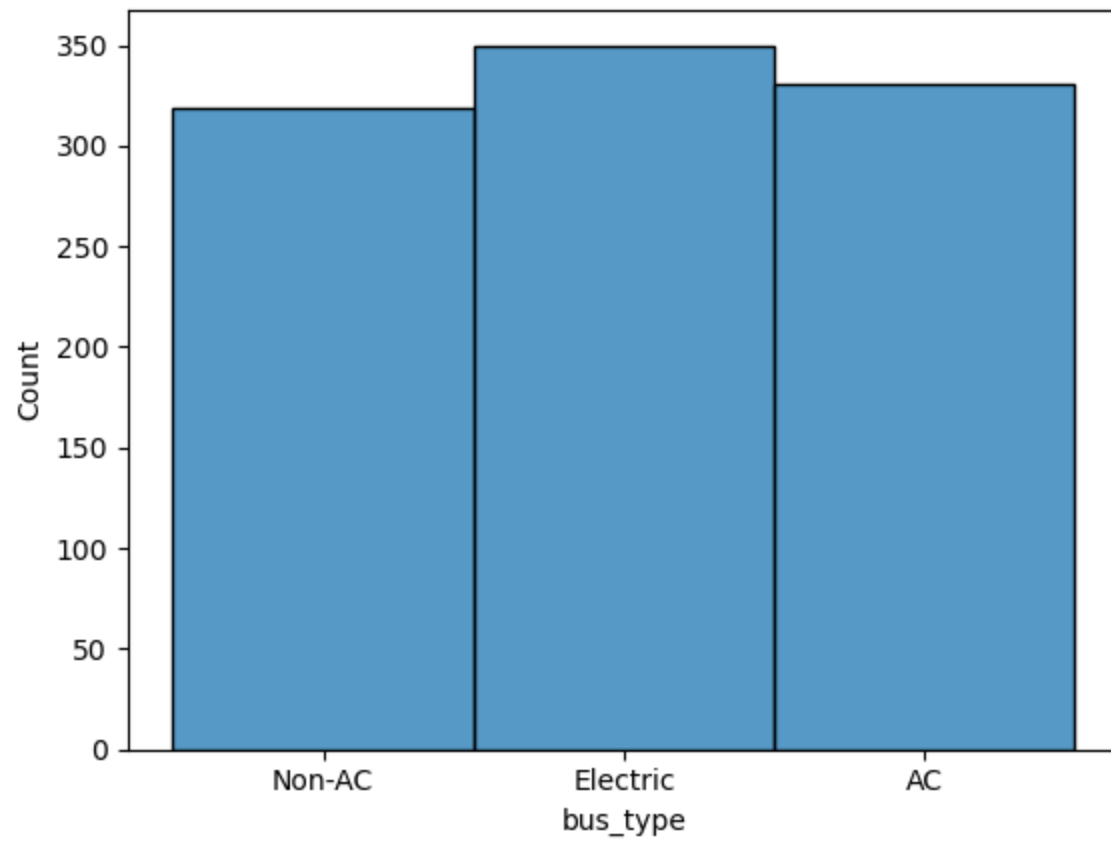
✓ 0.4s



Click to add a breakpoint 'bus_type'],bins=30)

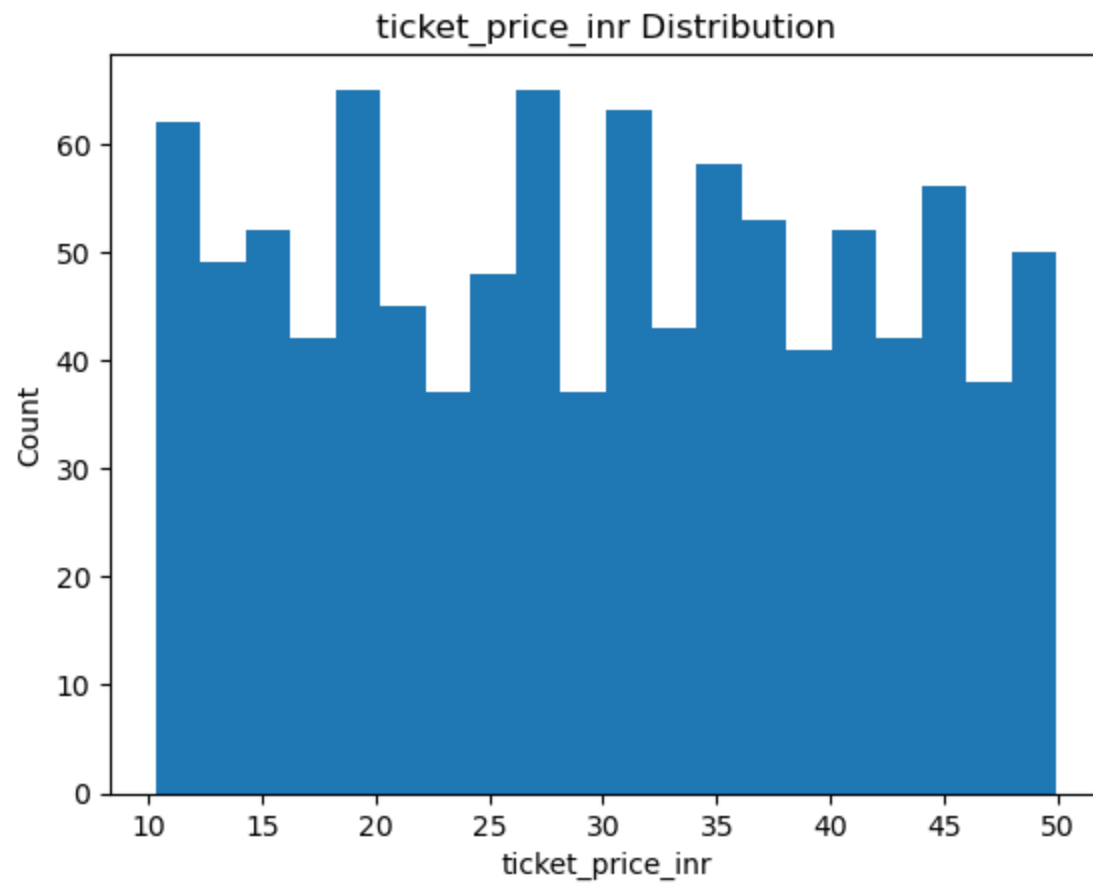
```
2 plt.show()
```

✓ 0.2s

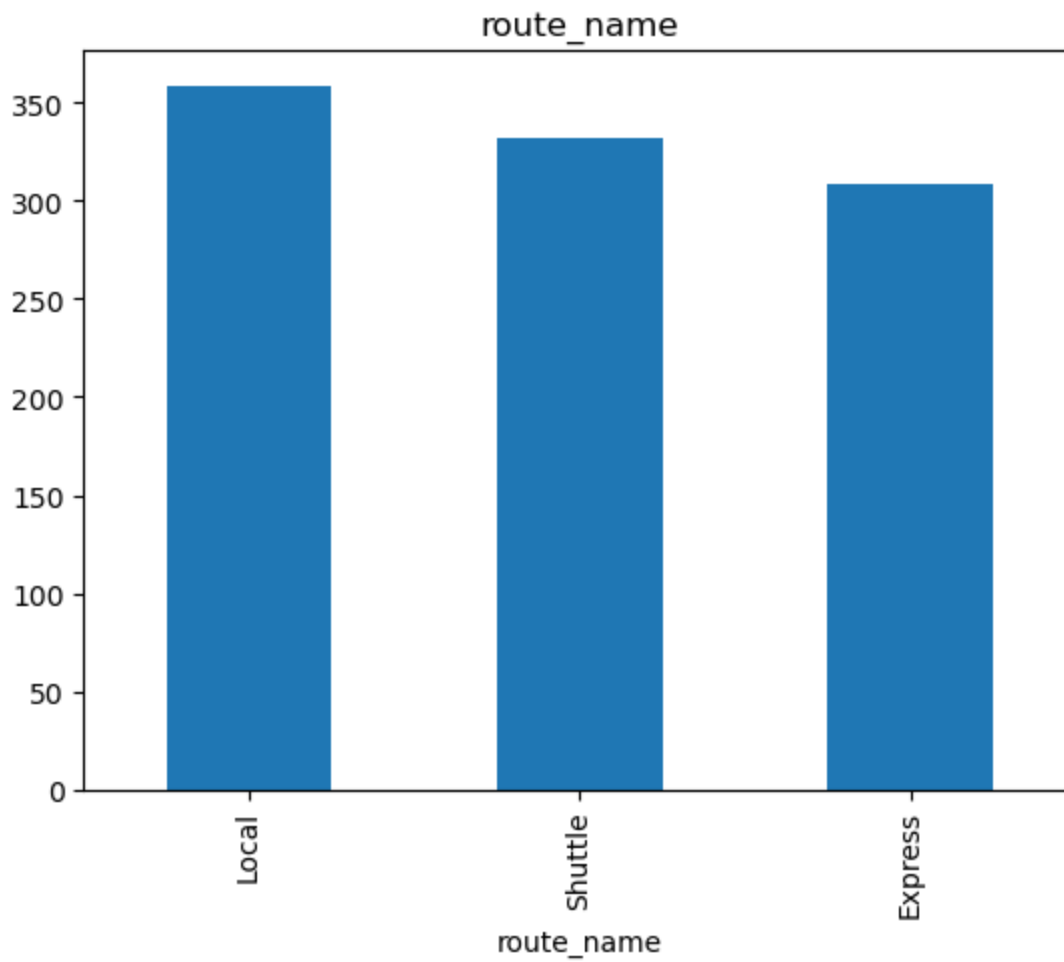


```
1 plt.hist(df["ticket_price_inr"], bins=20)
2 plt.title("ticket_price_inr Distribution")
3 plt.xlabel("ticket_price_inr")
4 plt.ylabel("Count")
5 plt.show()
```

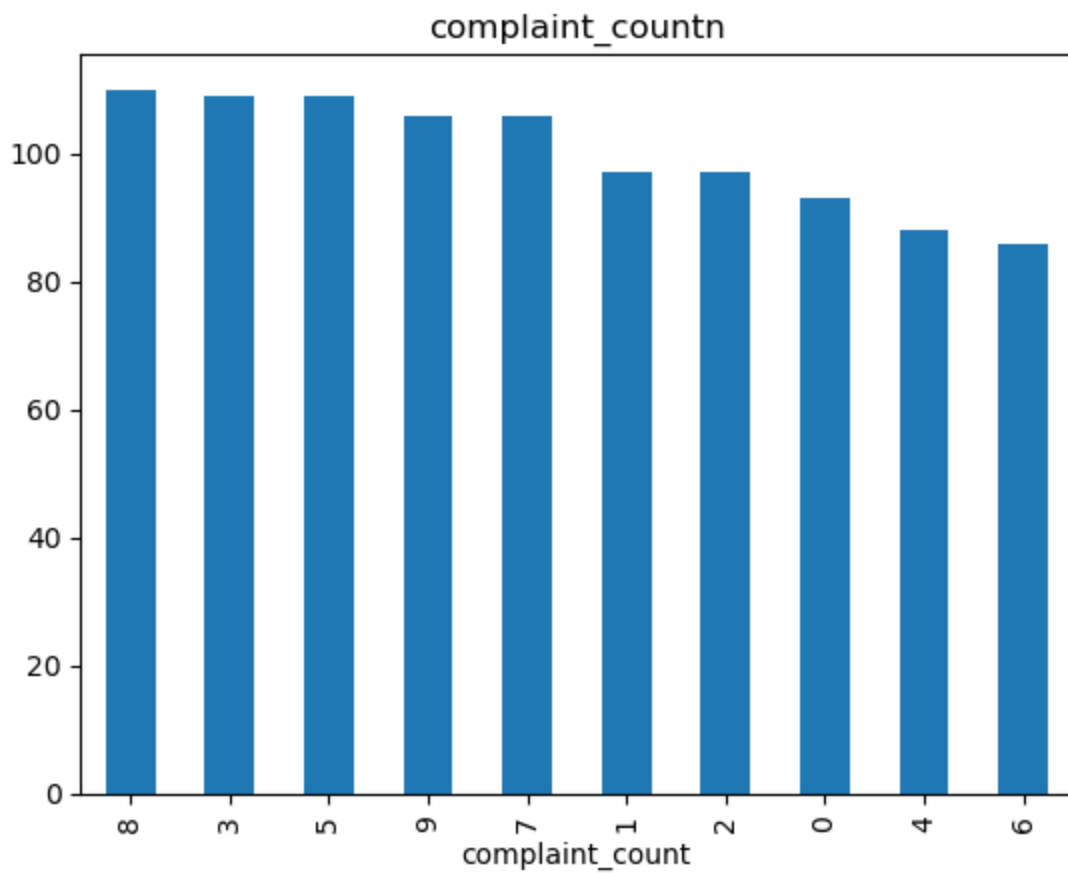
✓ 0.3s



```
1 df["route_name"].value_counts().plot(kind="bar")
2 plt.title("route_name")
3 plt.show()
✓ 0.2s
```

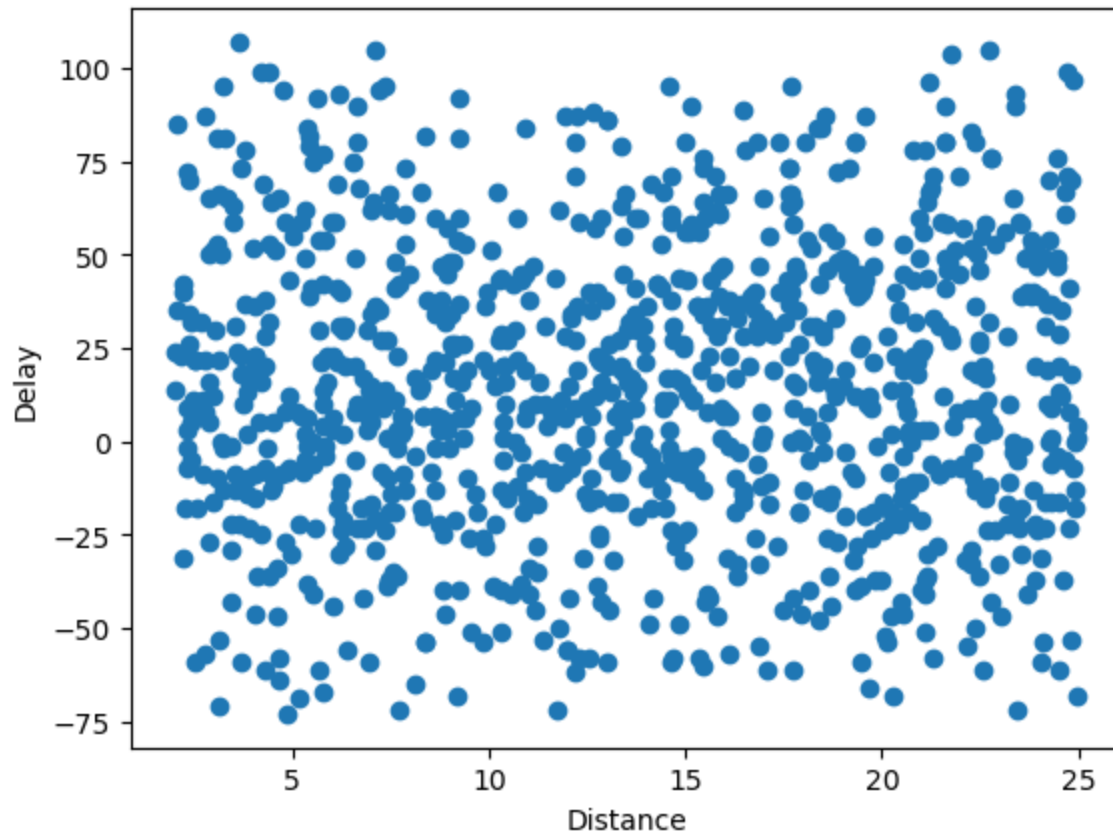


```
1 df["complaint_count"].value_counts().plot(kind="bar")
2 plt.title("complaint_countn")
3 plt.show()
✓ 0.2s
```

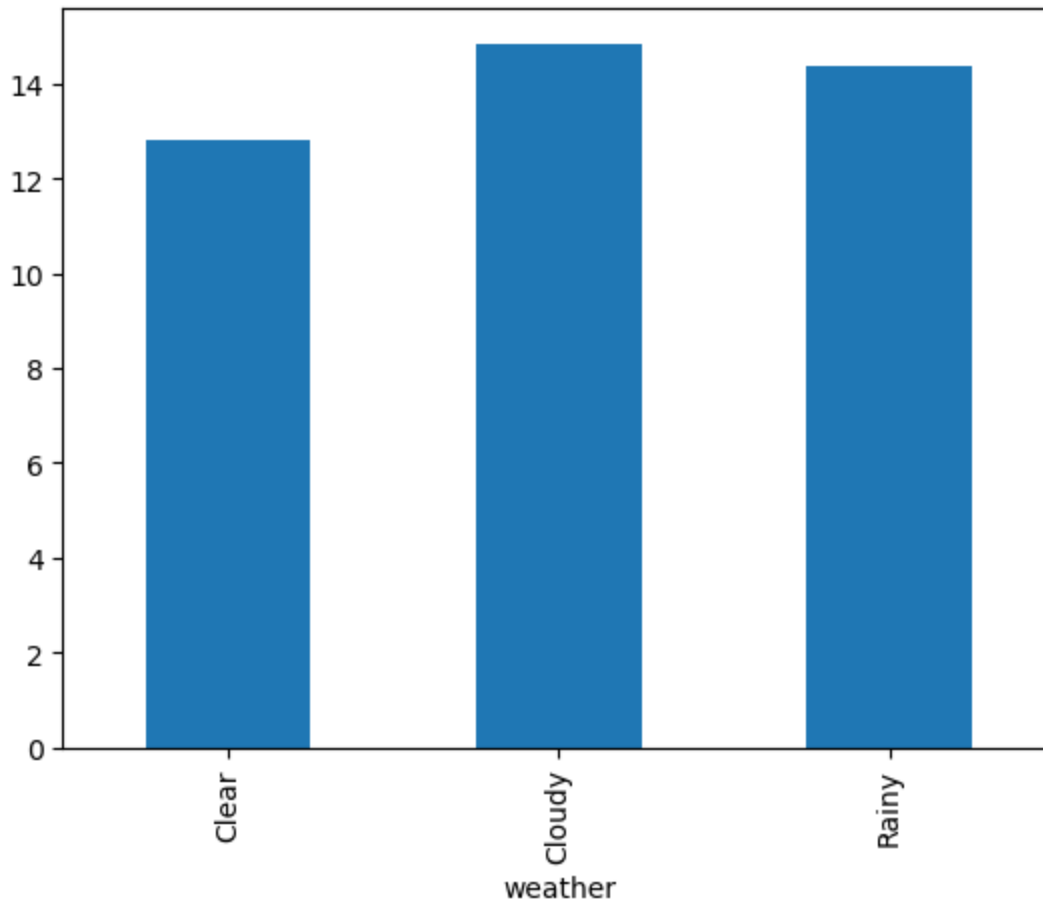



```
1 plt.scatter(df["distance_km"], df["delay_min"])
2 plt.xlabel("Distance")
3 plt.ylabel("Delay")
4 plt.show()
```

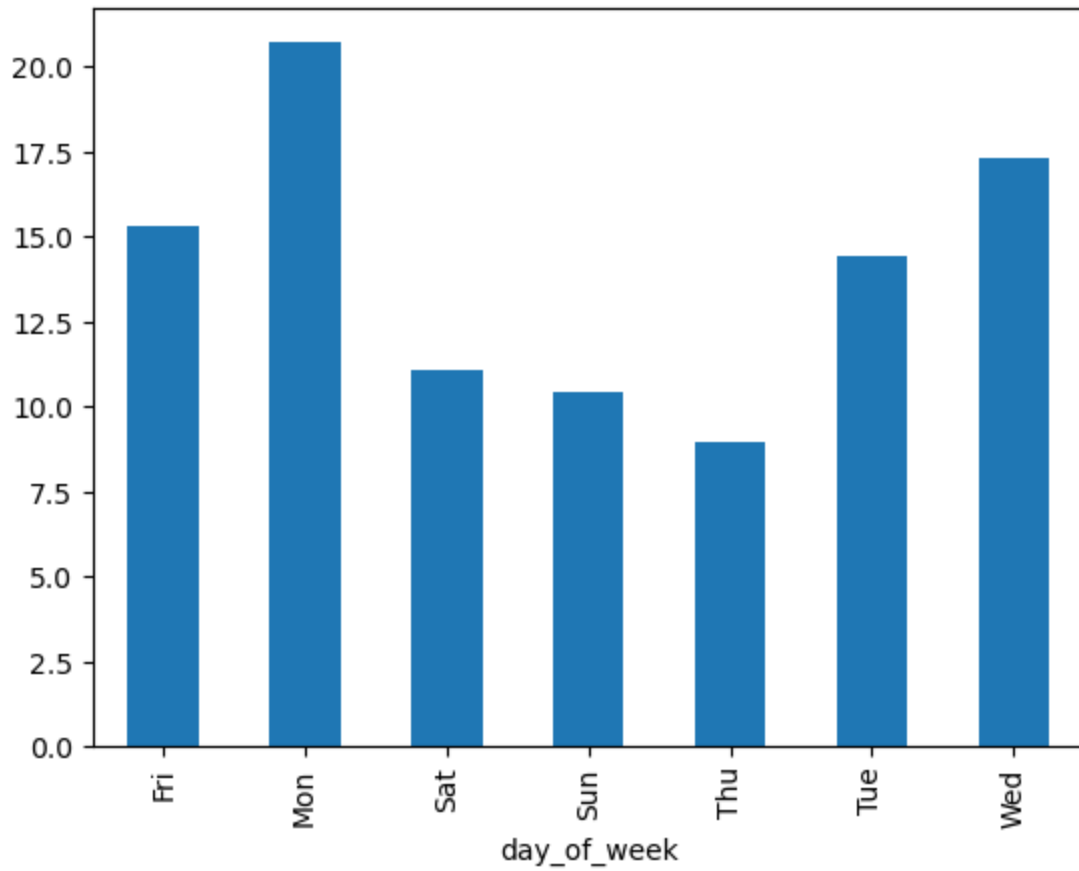
✓ 0.2s



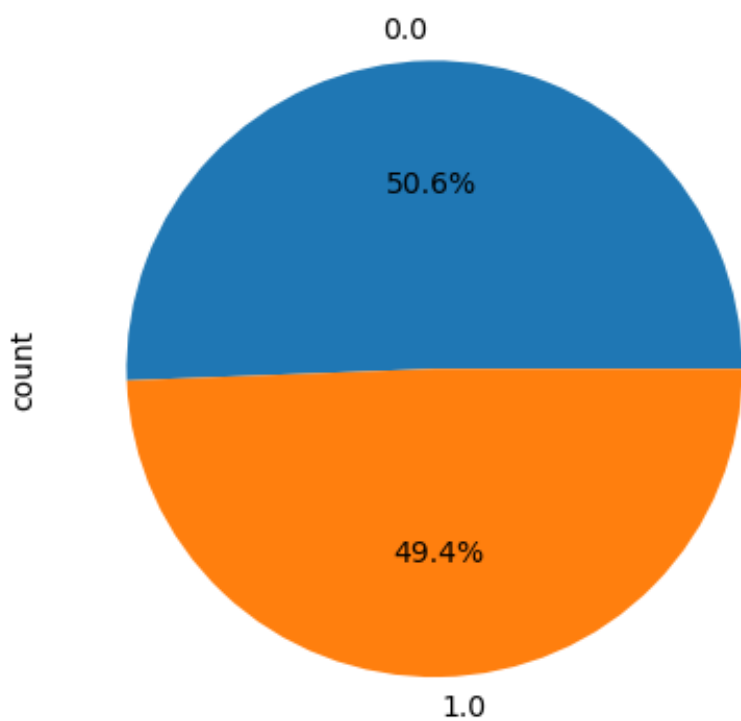
```
1 df.groupby("weather")["delay_min"].mean().plot(kind="bar")
2 plt.show()
✓ 0.6s
```



```
Click to add a breakpoint .of_week")["delay_min"].mean().plot(kind="bar")
2 plt.show()
✓ 0.1s
```

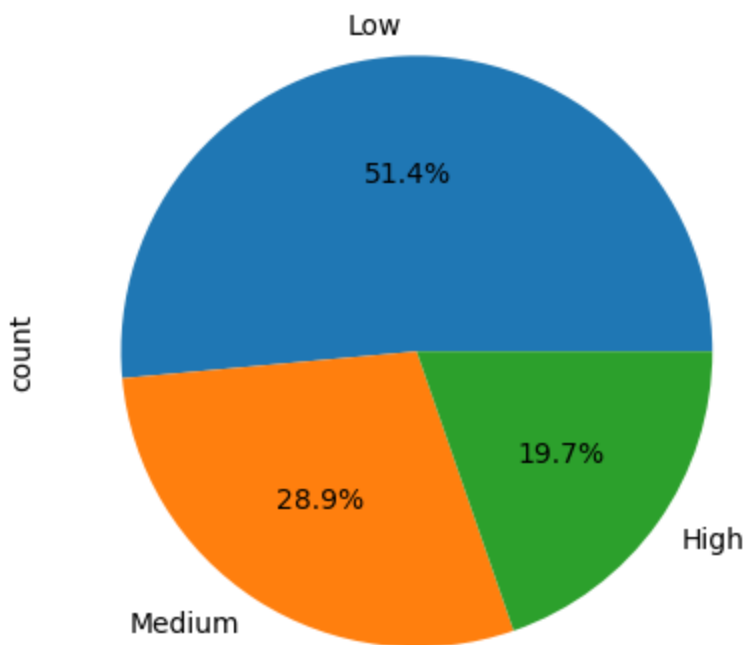


```
1 df["peak_hour_flag"].value_counts().plot(kind="pie", autopct="%1.1f%%")
2 plt.show()
✓ 0.1s
```



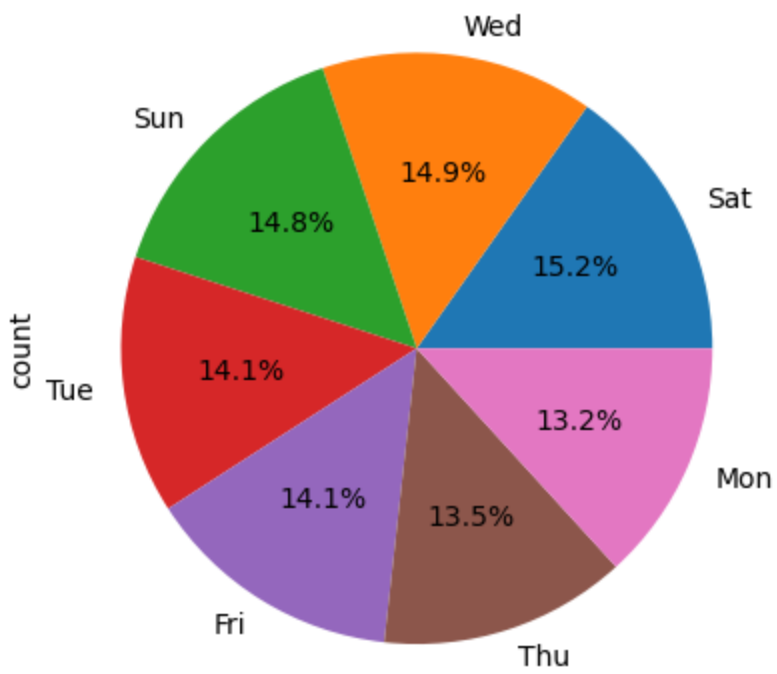
```
1 df["risk_level"].value_counts().plot(kind="pie", autopct="%1.1f%%")
2 plt.show()
```

✓ 0.1s



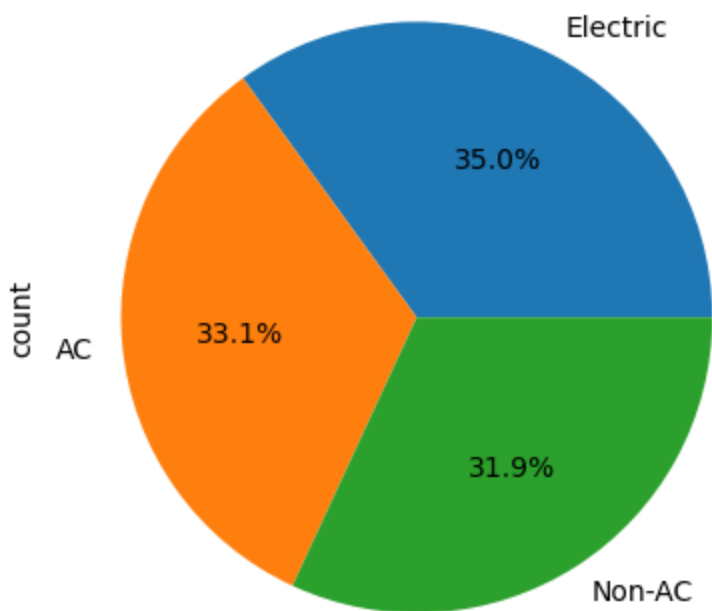
```
1 df["day_of_week"].value_counts().plot(kind="pie", autopct="%1.1f%%")
2 plt.show()
```

✓ 0.1s



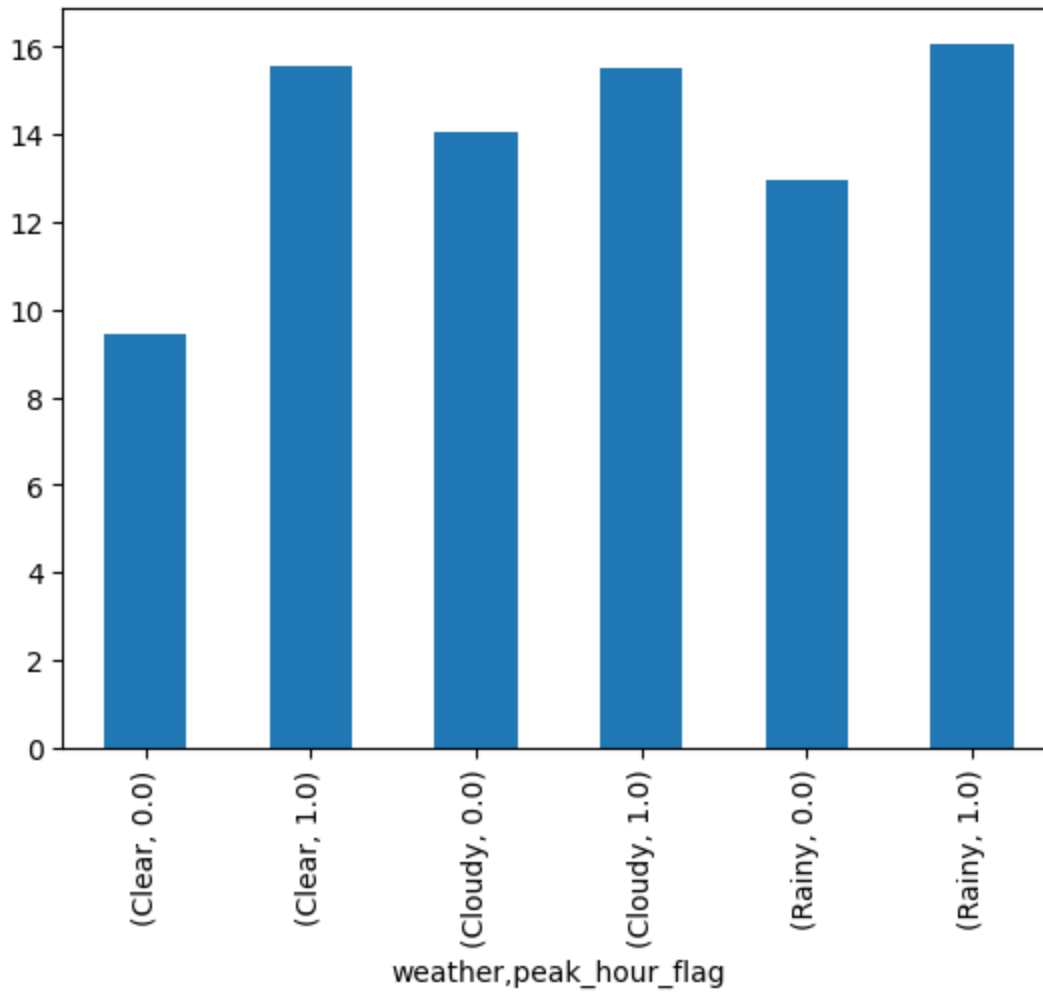
```
1 df["bus_type"].value_counts().plot(kind="pie", autopct="%1.1f%%")
2 plt.show()
```

✓ 0.1s

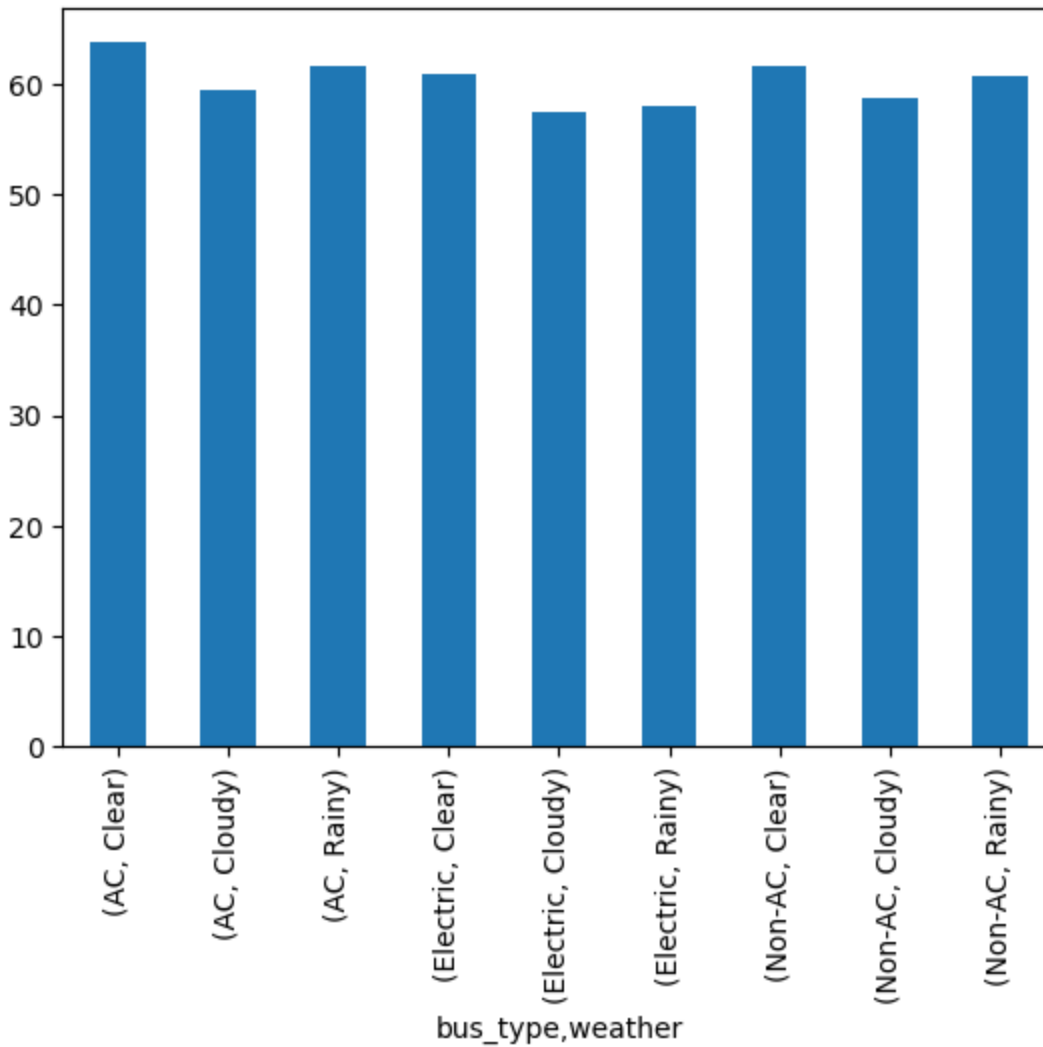


```
1 df["weather"].value_counts().plot(kind="pie", autopct="%1.1f%%")
2 plt.show()
```

✓ 0.1s



```
1 df.groupby(["bus_type","weather"])["occupancy_percent"].mean().plot(kind="bar")
2 plt.show()
✓ 0.3s
```

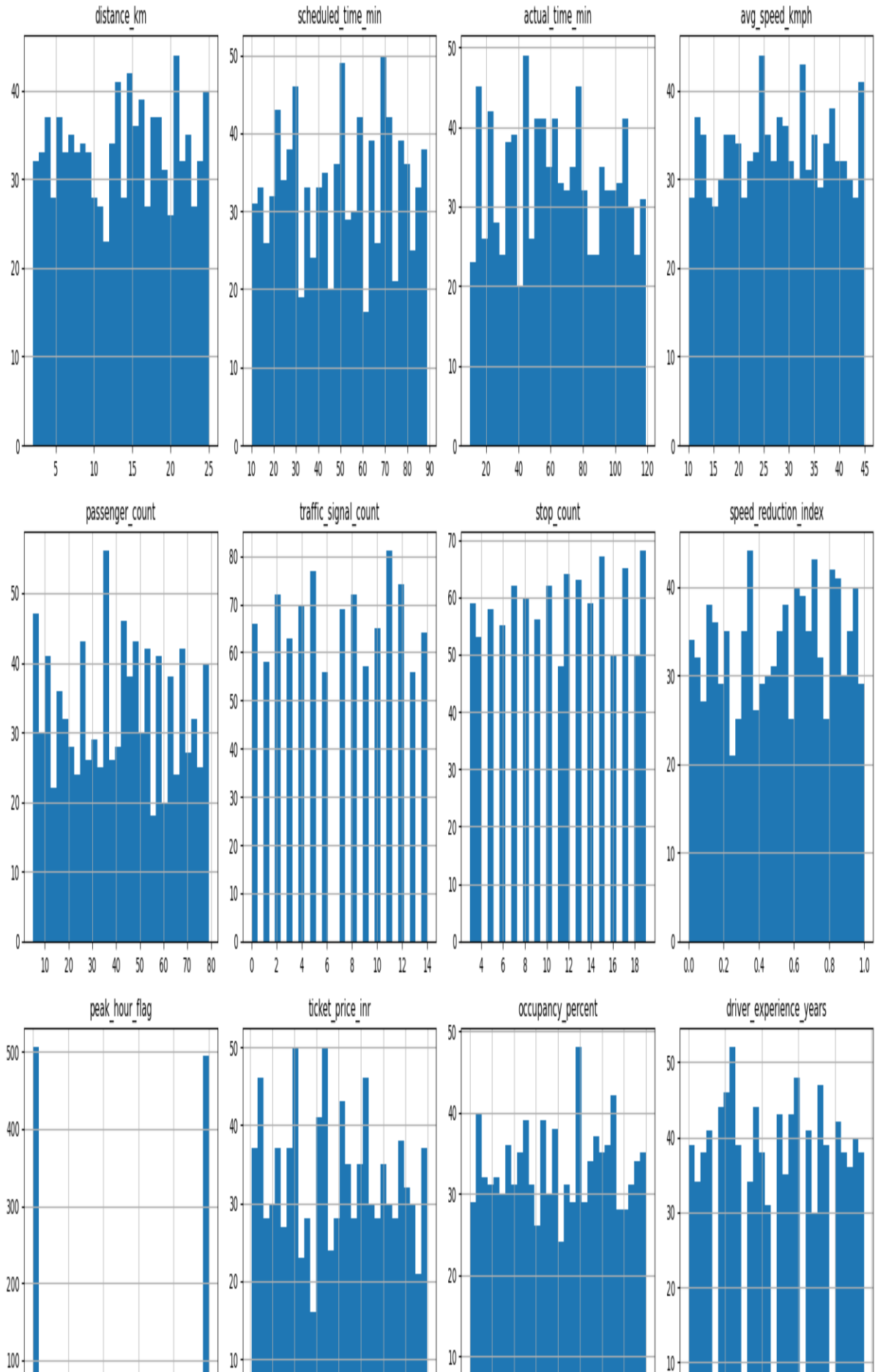


Numerical Feature Distribution

```
1 num_cols = df.select_dtypes(include=np.number).columns
2
3 df[num_cols].hist(figsize=(18,14),bins=30)
4 plt.suptitle('Numerical Feature Distributions')
5 plt.tight_layout()
6 plt.show()
```

✓ 3.8s

Numerical Feature Distributions

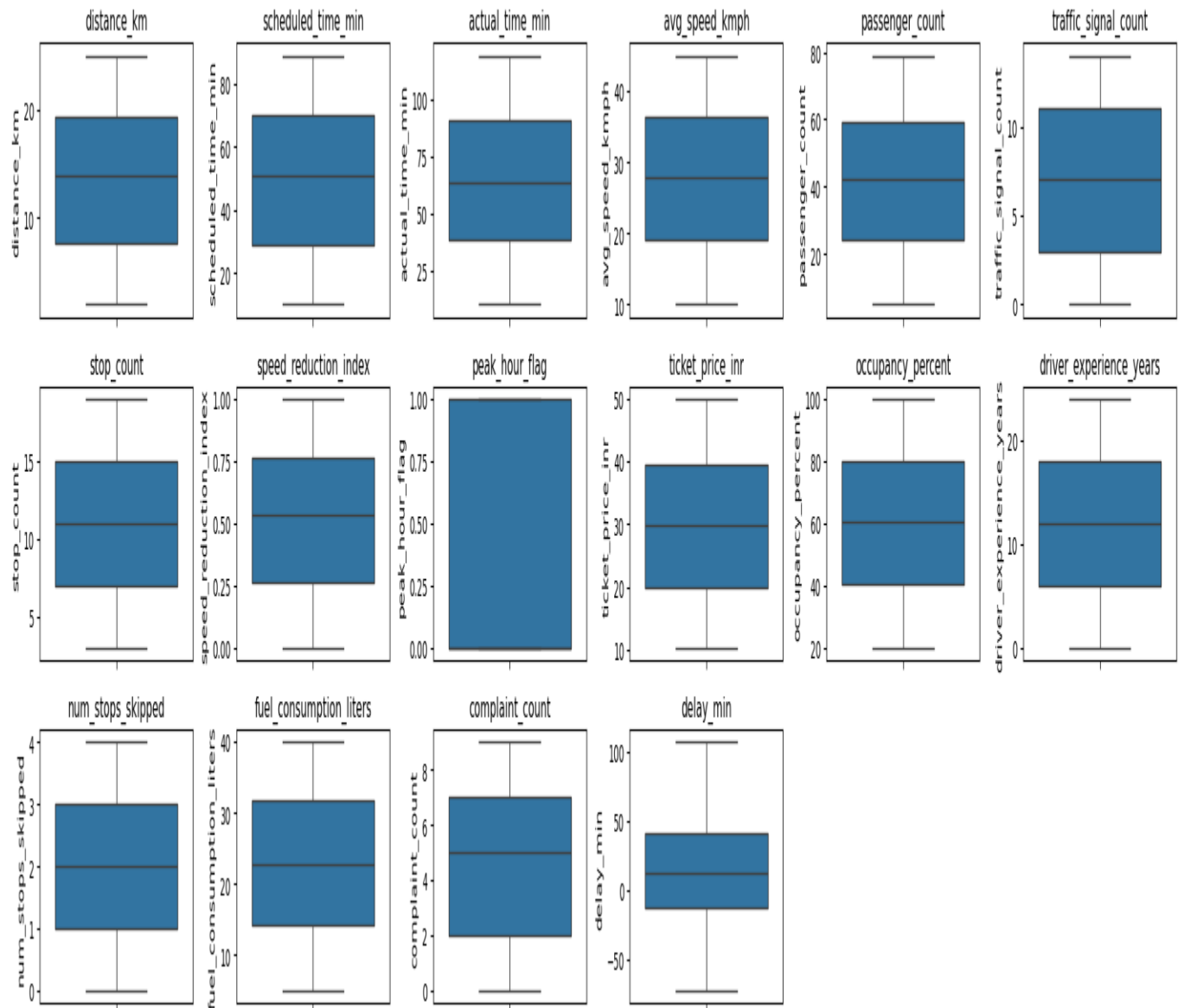


Boxplots for all numeric columns

Genera

```
1 num_cols = df.select_dtypes(include=np.number).columns
2
3 plt.figure(figsize=(20,12))
4 for i,col in enumerate(num_cols):
5     plt.subplot(6,6,i+1)
6     sns.boxplot(y=df[col])
7     plt.title(col)
8 plt.tight_layout()
9 plt.show()
```

✓ 3.5s

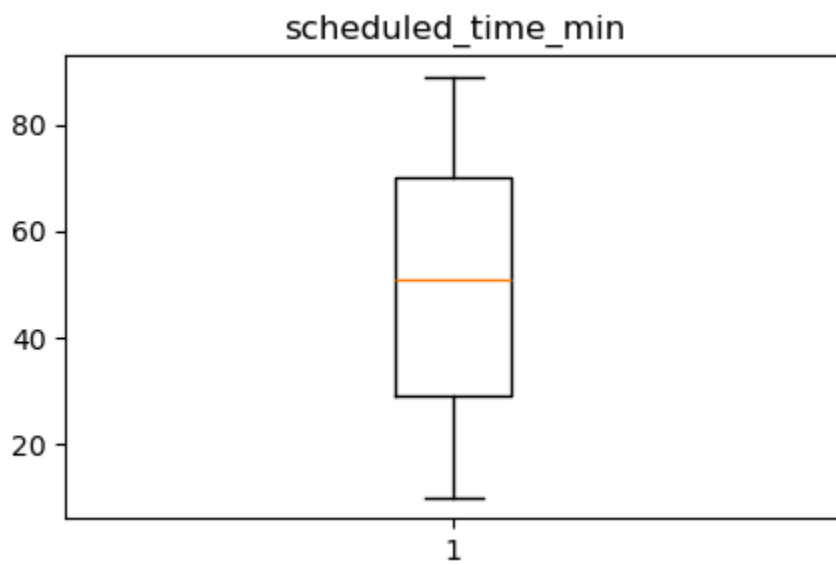
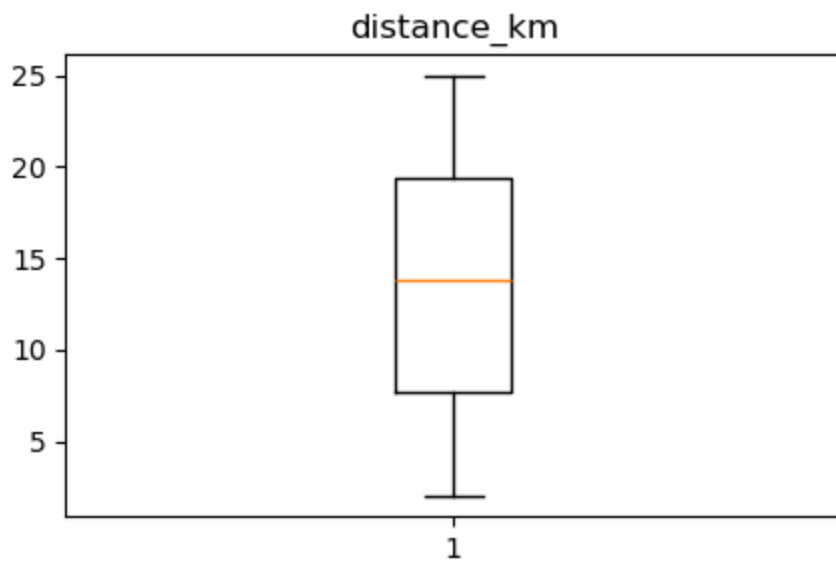


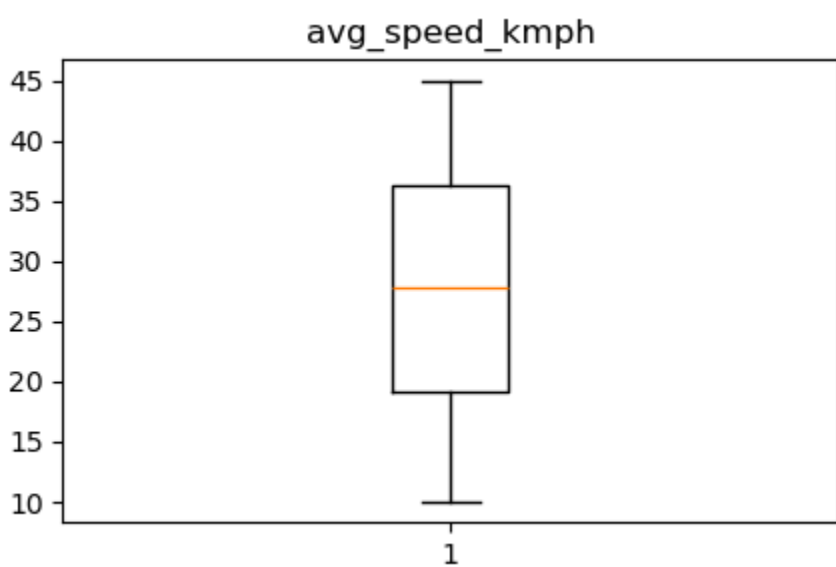
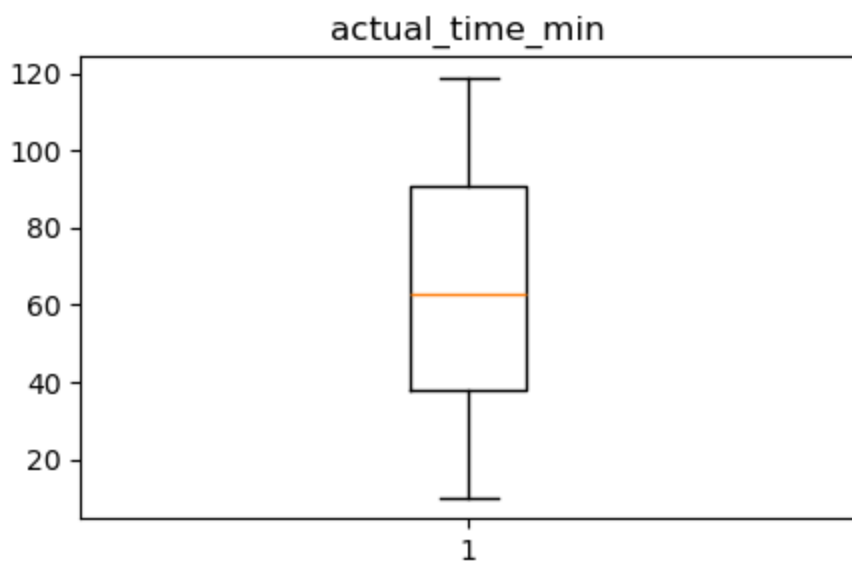
```

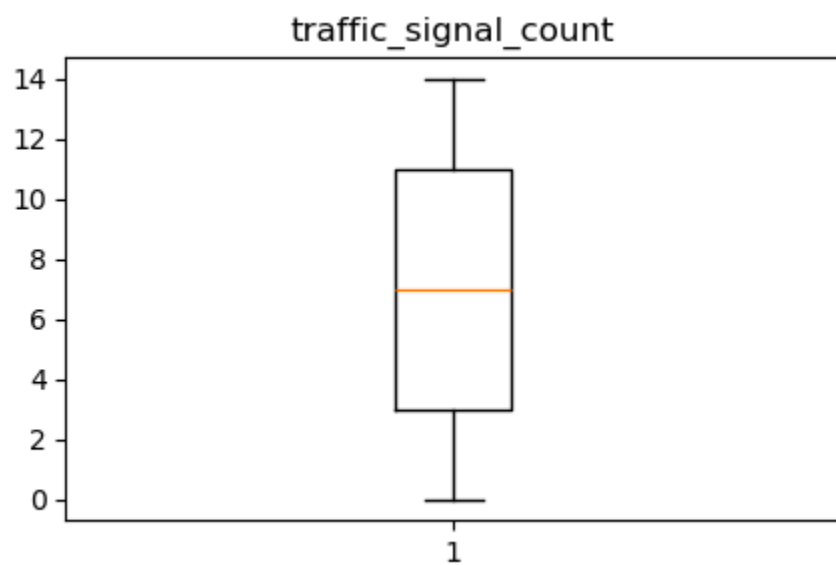
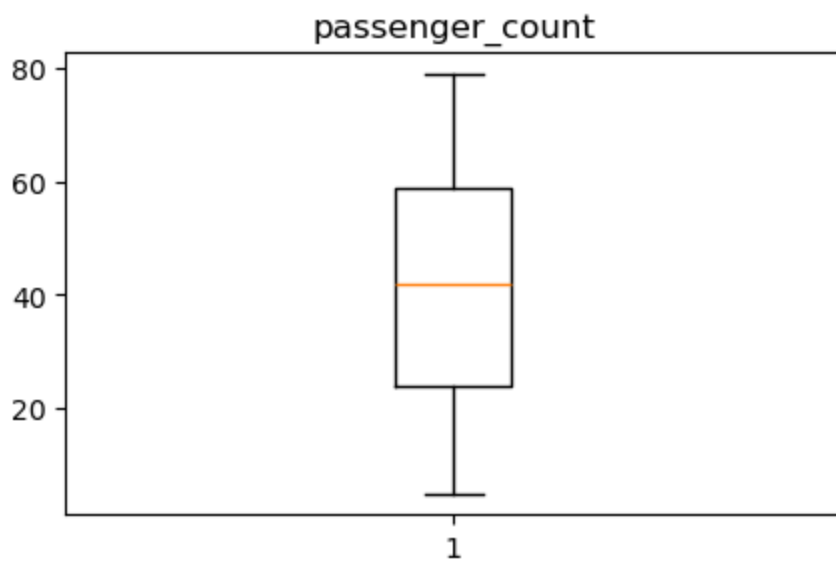
1 # Select only numeric columns
2 num_cols = df.select_dtypes(include=np.number).columns
3
4 for col in num_cols:
5     plt.figure(figsize=(5,3))
6     plt.boxplot(df[col].dropna())
7     plt.title(col)
8     plt.show()

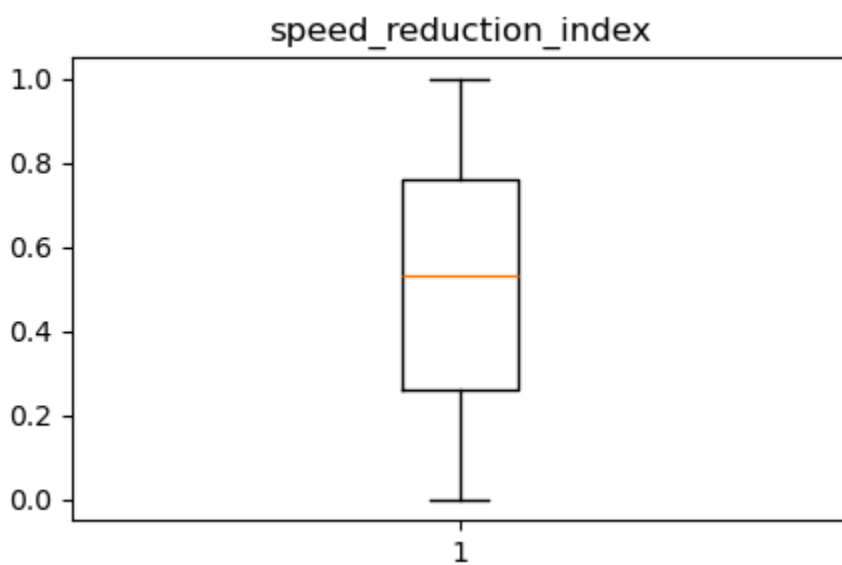
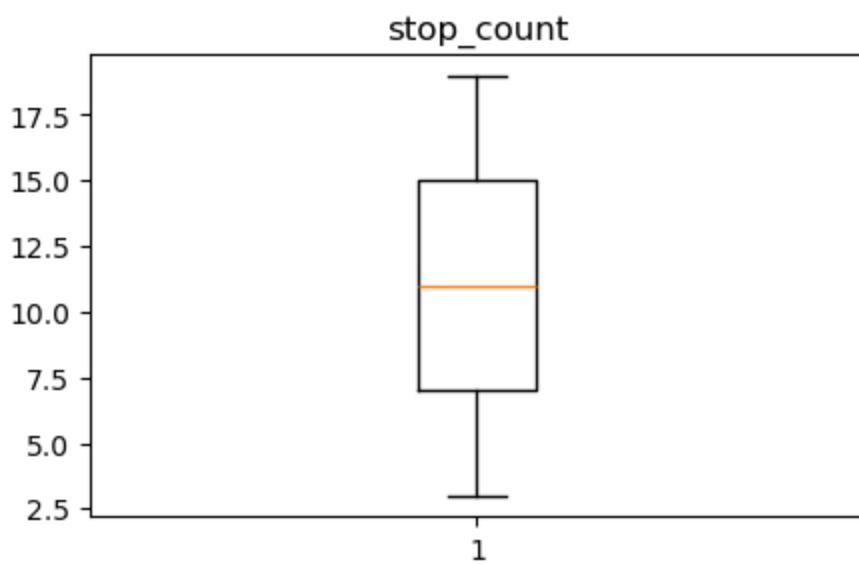
```

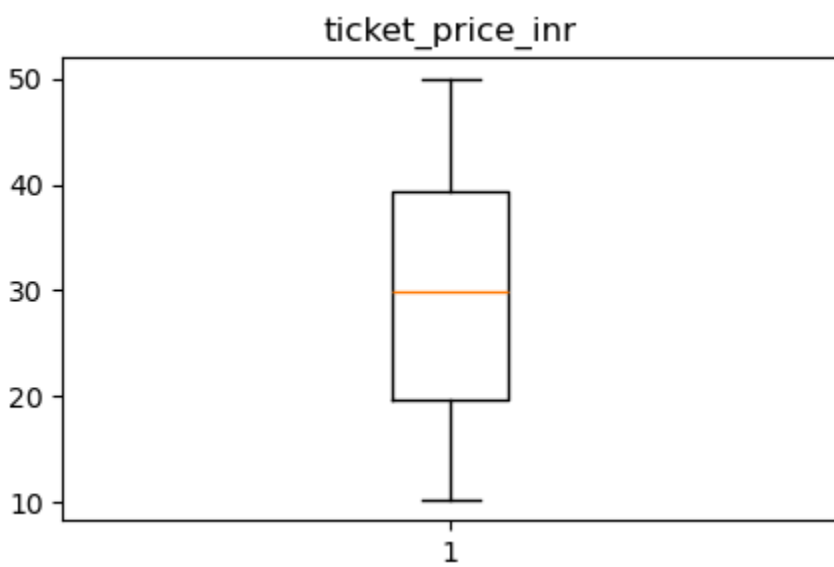
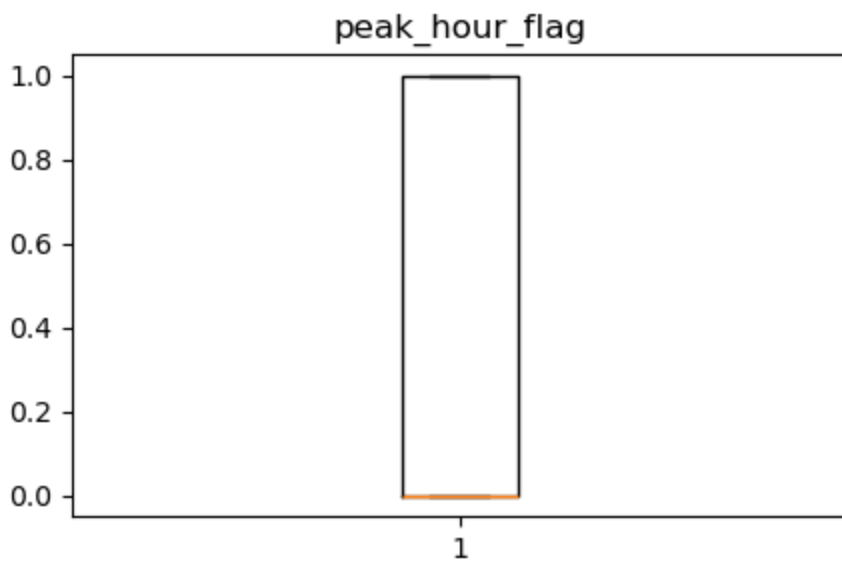
✓ 3.6s

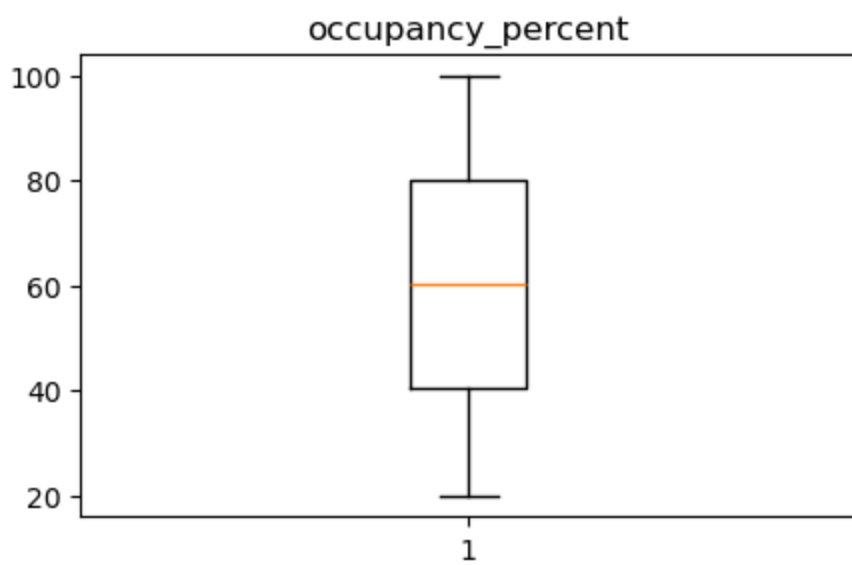


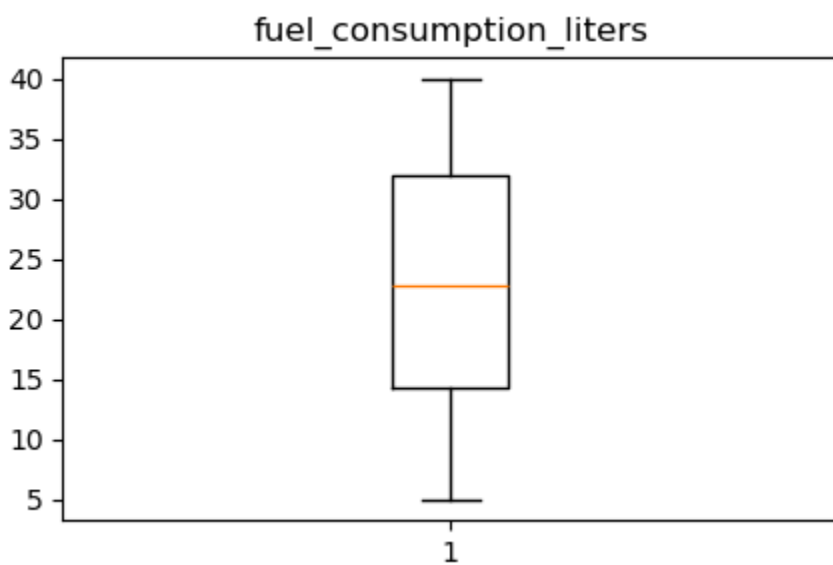
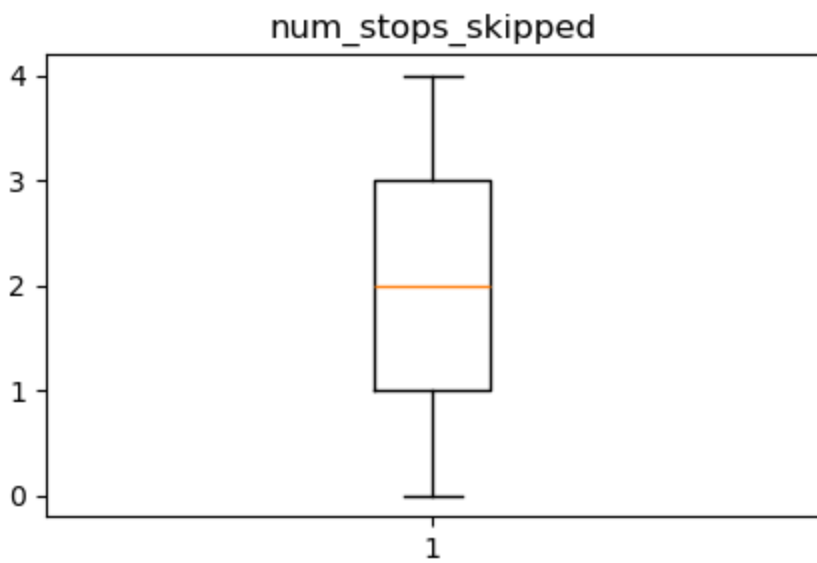


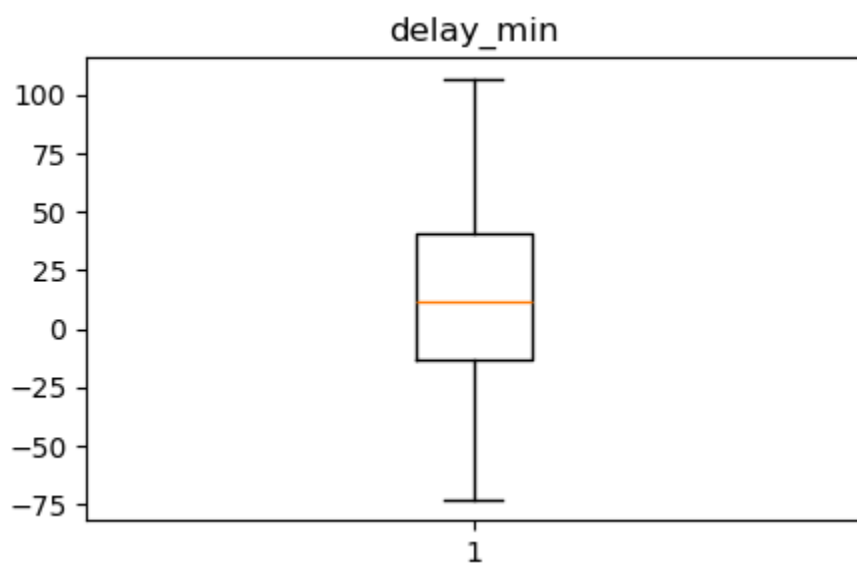
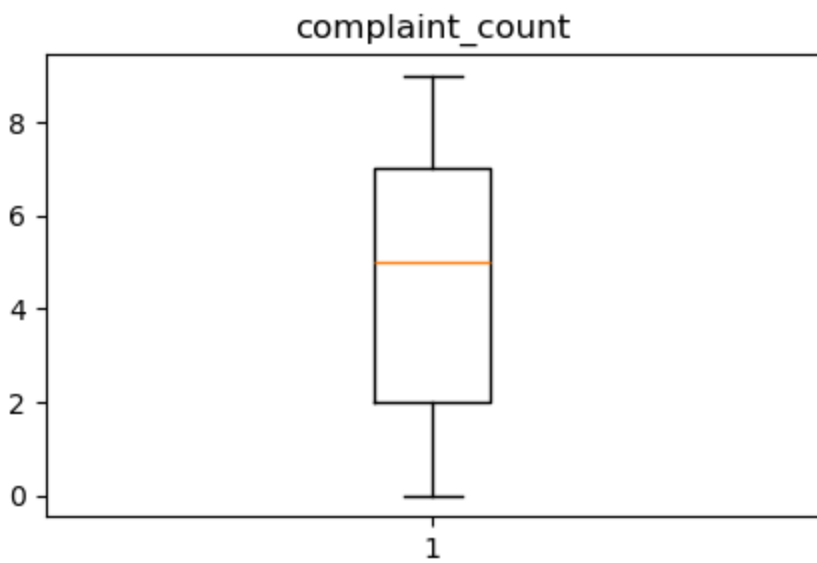






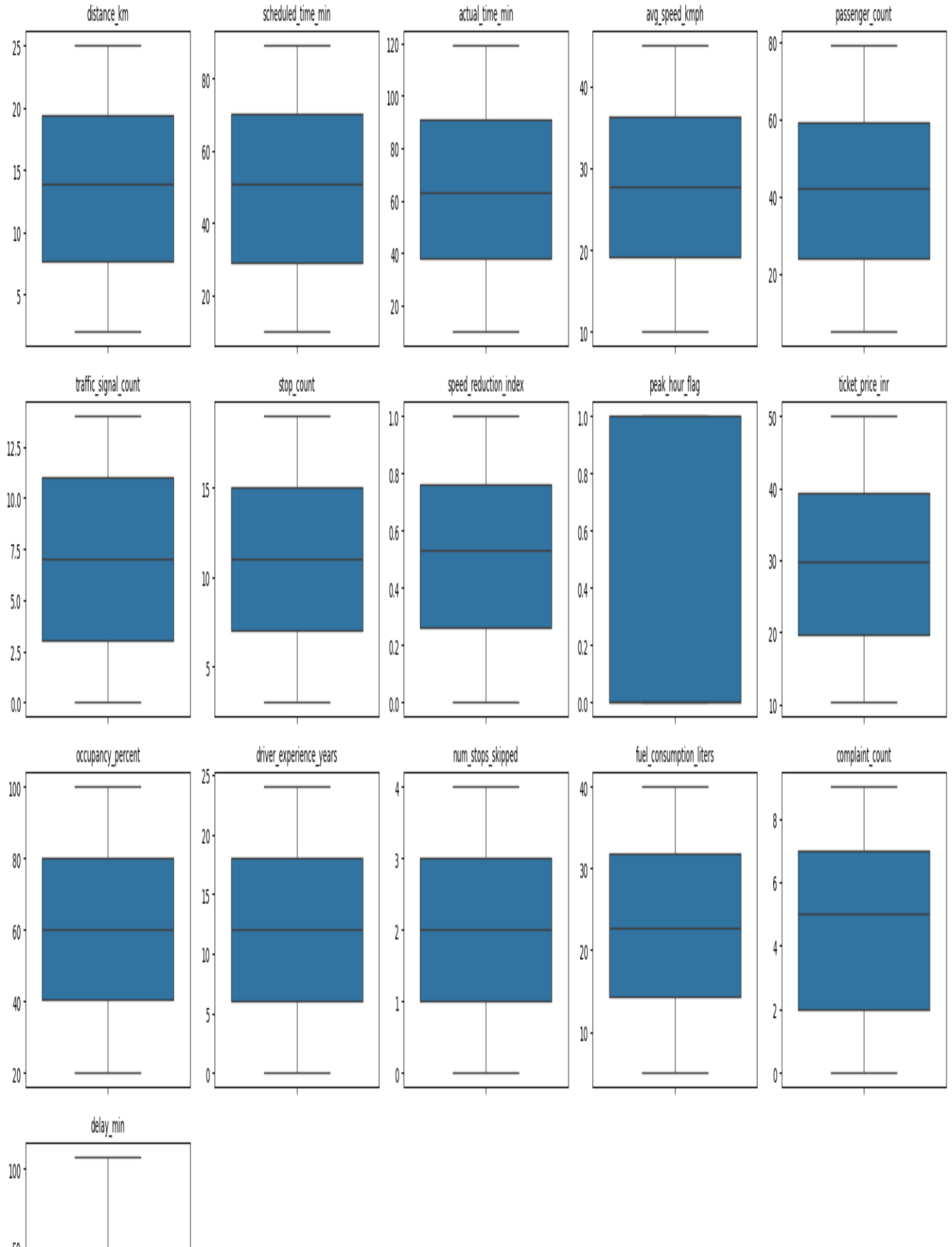






```
1 # Create boxplots
2 plt.figure(figsize=(20, 14))
3
4 for i, col in enumerate(num_cols):
5     plt.subplot(5, 5, i + 1)
6
7     sns.boxplot(y=df[col])
8
9     plt.title(col, fontsize=10)
10    plt.ylabel("")
11
12 plt.suptitle(
13     "Outlier Detection Using Boxplots (IQR Method)",
14     fontsize=18,
15     fontweight="bold"
16 )
17
18 plt.tight_layout(rect=[0, 0, 1, 0.96])
19
20
21
22
23 plt.show()
```

Outlier Detection Using Boxplots (IQR Method)




```

1 num_cols = df.select_dtypes(include="number").columns
2
3 for col in num_cols:
4     Q1 = df[col].quantile(0.25)
5     Q3 = df[col].quantile(0.75)
6
7     IQR = Q3 - Q1
8
9     lower = Q1 - 1.5 * IQR
10    upper = Q3 + 1.5 * IQR
11
12    outliers = df[(df[col] < lower) | (df[col] > upper)]
13
14    print(col, ":", len(outliers))

```

```

... distance_km : 0
scheduled_time_min : 0
actual_time_min : 0
avg_speed_kmph : 0
passenger_count : 0
traffic_signal_count : 0
stop_count : 0
speed_reduction_index : 0
peak_hour_flag : 0
ticket_price_inr : 0
occupancy_percent : 0
driver_experience_years : 0
num_stops_skipped : 0
fuel_consumption_liters : 0
complaint_count : 0
delay_min : 0

```

```

1 df.to_csv("cleaned_bus_transport.csv", index=False)
5] ✓ 0.0s

```

